Reducing the Cost of LLM Coding and Reasoning Models with Routers

Anonymous authors
Paper under double-blind review

Abstract

Routers typically choose a model per query by trading predicted correctness against cost. However, most price each model at a fixed cost, assuming that the number of tokens each model outputs does not vary significantly. We find that the output length of reasoning models varies by an order of magnitude across queries, and ask when per-query cost is worth predicting, what a prediction captures, and how cheaply it can be had. We predict **both** success probability **and** route-specific output length from the prefill activations of a single small frozen 4B LLM, using a linear readout per route. On 6,500 MMLU-Pro, 1,000 Omni-MATH, and 341 LiveCodeBench tasks, predicted costs reduce spending at matched accuracy by 23.4%, 31.5% and 27.8% relative to median-length pricing, averaged over ten random training sets. Perfect per-query cost knowledge would save 40–58%, so our readouts capture two fifths to over two thirds of the achievable saving, and they beat every alternative estimator we evaluated, including a text-embedding model of the prefill's size. The router does not beat the most accurate single route, deepseek-v4-flash, at that route's accuracy; its savings lie below it. In mixed traffic over all nine benchmarks, one readout saves 22% beyond a router that knows each query's benchmark, and 10% beyond one that also prices from difficulty within the benchmark. Across nine pools the saving follows two conditions: output length must vary within a route more than prices differ between routes, which holds for reasoning models but much less for non-reasoning models on the same problems, and that variation must be legible from the prompt. Pricing from difficulty alone matches the readouts where length tracks difficulty. The dedicated readout is what generalises: it keeps most of its saving with 20 labelled problems, when a new model is added from 10 examples, and, on four of seven target benchmarks, when a router trained on other benchmarks is applied to a new one without labels; in these settings text-embedding and item-response estimators fall behind it. Every small LLM prefill we tried (0.6–4B, four model families) can be used as a router in this way, and fully fine-tuning a small model on the same labels does not match a linear readout of a frozen one.

1 Introduction

Routing reasoning models requires predicting both correctness and cost before generation. Token rates are published; the number of tokens is not. A model that is cheap on average can be expensive on a particular problem: on LiveCodeBench, a single model's output length differs 11–121× between its 10th- and 90th-percentile problems (Fig. 1).

Prefill routers predict correctness from a small encoder's prompt activations; the prefill router of Varshney et al. (Varshney et al., 2026) explicitly prices outputs at median training length. We read output length from the same activations (Fig. 2). Cost prediction adds one linear readout per route to an existing prefill router, without another encoder pass.

This raises three questions, which organise the paper. When is per-query cost worth predicting? What does a prediction of it capture? And how cheaply can a router obtain it: with how many labels, for how many new models, and on how many new benchmarks?

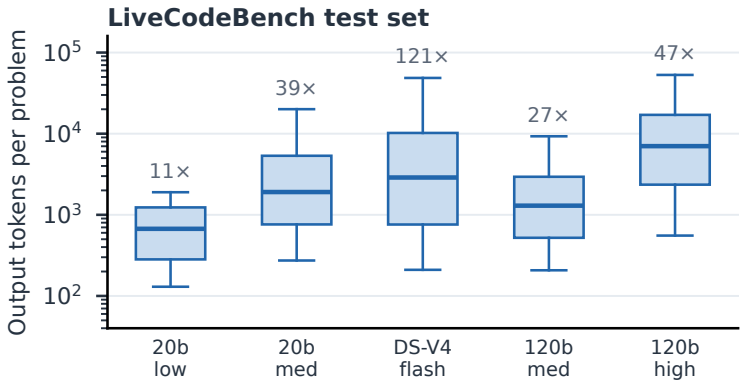


Figure 1: **Output length varies by orders of magnitude within one model.** Per-problem output tokens by route on the LiveCodeBench test set (log scale). Boxes: interquartile range; whiskers: 5th–95th percentile; line: median; above each box: 90th/10th-percentile ratio.

We contribute: (i) a measurement of what per-query cost knowledge is worth on nine pools, under one protocol at billed prices (29–61% of spending at matched accuracy wherever reasoning lengths vary), and on the same problems with non-reasoning routes, where it is worth much less and no estimator we tried captures much of it (Sections 5 and 8); (ii) prefill cost readouts that reduce spending by 23.4–31.5% relative to median pricing on three large test sets and outperform mean-length, prompt-feature, text-embedding and ZeroRouter cost estimators (Table 1); (iii) a mechanism: per-query length is mostly a problem-level property shared across routes; where it tracks difficulty, pricing from the success readouts alone is enough, and where it tracks the work a question requires, only a dedicated readout captures it (Section 6). The two failures we find have identifiable causes: too little headroom, or headroom that no estimator we tried reads from the prompt (Section 8); (iv) generalisation: the dedicated readout keeps most of its saving with few labels, when a new model is onboarded from a handful of examples, and, on four of seven targets, when a router trained on other benchmarks is applied to a new one without labels; the external estimators do not, and pricing from difficulty transfers only between benchmarks whose difficulty means the same kind of work (Section 7); (v) mixed traffic: on one mixture of all nine benchmarks, the cost readout saves 22.4% beyond pricing by benchmark identity, the signal recent work finds explains much of what quality routers gain (Li et al., 2026a; Lee, 2026), and 9.9% beyond pricing by benchmark and difficulty together (Section 7); (vi) controls showing the gain is the prefill representation: it holds for every small prefill we tried (0.6–4B, four families), a text-embedding model of the same size reads cost worse under every readout, fully fine-tuning a small model on the same labels does not match a linear readout of the frozen prefill, and our linear readouts on fixed layers match the prefill router's full success pipeline on four of five sets. As a side note, the same model's provider endpoints drift; the readouts can absorb this with one offset per endpoint, and we pin providers instead.

Related work. Learned routers pick one model per query from the prompt, trading predicted quality against price: RouteLLM learns a strong-versus-weak decision from preference data (Ong et al., 2024), RouterBench standardizes the evaluation (Hu et al., 2024), and IRT-Router models query difficulty and model ability with item response theory (Song et al., 2025). Closest to us, the prefill router of Varshney et al. predicts each model's correctness from a small encoder's prompt activations (Varshney et al., 2026), and pre-generation activations are known to encode whether a model will fail (Lugoloobi et al., 2026). These routers, like SWE-Router for agentic software engineering (Son et al., 2026), price each model at a per-model constant; the prefill router uses the median training output length, which is our main reference. Fewer routers predict per-query cost: MixLLM regresses output length on query embeddings (Wang et al., 2025), CARROT uses embedding nearest neighbours or a fine-tuned RoBERTa (Somerstep et al., 2025), GraphRouter predicts cost on a query–model graph (Feng et al., 2024), Route-To-Reason predicts reasoning-model length from text embeddings (Pan et al., 2025), ZeroRouter prices queries by item-response difficulty bins (Yan et al., 2026), and SCOPE estimates a query's cost from retrieved past behaviour (Cao et al.,

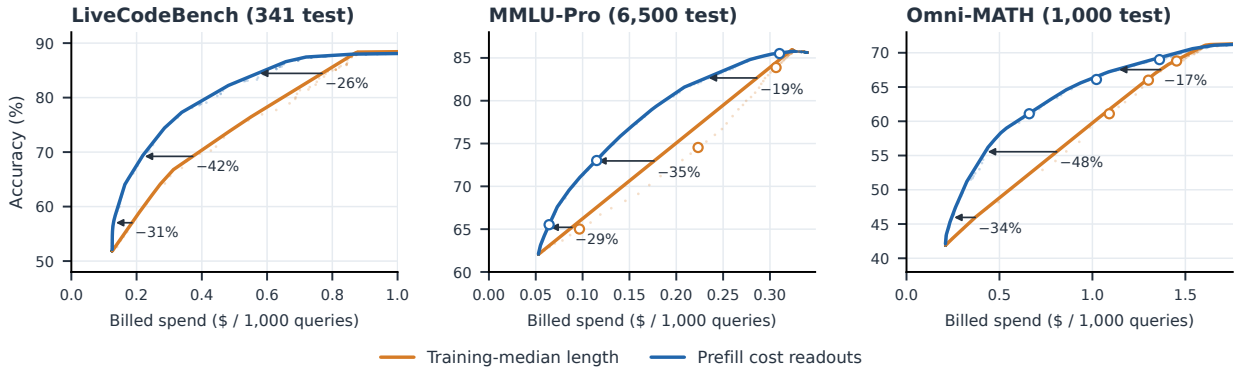


Figure 2: **Cost–accuracy frontiers at billed prices.** Prefill cost readouts (blue) versus training-median length (orange), with the same success predictions and the same encoder pass. Dots: the deterministic policy at each V; lines: their frontiers; hollow circles: policies selected on calibration for fixed accuracy targets. Arrows run from the median rule's frontier to ours at matched accuracy and give the cost saved. One training set is shown (the manifest split); averaged over the shared accuracy band and over ten training sets, the saving is 28% on LiveCodeBench, 23% on MMLU-Pro and 32% on Omni-MATH (Table 1).

2026). R2-Router routes over pairs of a model and an output-length budget, so length is controlled rather than predicted (Xue et al., 2026), and SCX Router reads task type, difficulty and an expected output length from a small decoder's KV cache as properties of the task, not of each candidate model (Stepanov et al., 2026). We compare against the embedding-based and ZeroRouter estimators directly, with every estimator given the same success predictions. Recent large-scale evaluations find that routers land close to an oracle that assigns each dataset its best model, and attribute much of their gain to coarse domain structure (Li et al., 2026a); a static assignment by task type closes most of the gap to the per-query oracle on a small benchmark (Lee, 2026). Both concern which model answers correctly; we measure what per-query cost is worth beyond the benchmark a query comes from.

Cascades instead call models in sequence and stop when an answer is accepted: FrugalGPT learns a scorer that decides whether to escalate (Chen et al., 2023), AutoMix uses self-verification (Aggarwal et al., 2023), and Dekoninck et al. unify routing and cascading (Dekoninck et al., 2024). A cascade pays for the cheap attempt on every query; we route once, before generation. Without a verifier, a pre-generation router beats the best cascade on four of five datasets, LiveCodeBench among them (Bouchard, 2026). Activation-based length prediction also supports serving and scheduling of a single model (Messenger, 2026; Xie et al., 2026; Wen et al., 2026); we predict the length of other models from one shared prefill. Hosted open-weight models vary across providers in accuracy, length and price (Li et al., 2026b; Lin et al., 2026; Gao et al., 2025), and FACET selects among providers of a fixed model per task from measured outcomes (He et al., 2026).

2 Cost Prediction and Routing

For query x and route m (a model at a fixed reasoning effort, served by an endpoint), let $p_m(x)$ be the probability of a correct answer and $c_m(x)$ the expected generation cost. One route is chosen before generation,

$$m^*(x; V) = \arg\max_m \{V\hat{p}_m(x) - \hat{c}_m(x)\}, \tag{1}$$

where V is the value of a correct answer; sweeping V trades accuracy for cost. Every query is answered once, with no verifier or fallback.

Readouts. We concatenate mean and last-token activations from eight layers of a frozen Qwen3-4B (Yang et al., 2025a) (Instruct-2507; Thinking-2507 on Omni), standardized on training data. Success readouts are L2-regularized logistic regressions with held-out Platt calibration; cost readouts are ridge regressions of log

mean output length, regularized by training-only cross-validation. Predicted length is exponentiated with residual smearing and matched to the route's training mean, then priced: $\hat{c}_m(x) = r_m^{\text{in}} n_m^{\text{in}}(x) + r_m^{\text{out}} \hat{\ell}_m(x)$.

3 Experimental Protocol

The routes are gpt-oss-20b at low and medium effort (OpenAI, 2025), deepseek-v4-flash (DeepSeek-AI, 2026), and gpt-oss-120b at medium and high effort, called through OpenRouter, with deepseek-v4-flash pinned to one provider (StreamLake) throughout, because its output length depends on the serving provider (Appendix E). LiveCodeBench (Jain et al., 2024) has 892 problems (contests from September 2023 to April 2025) split at random into 441 training, 110 calibration and 341 test problems; we do not split by date, since contamination concerns the routed models, which are the same in every arm, not the router. Omni-MATH (Gao et al., 2024) and MMLU-Pro (Wang et al., 2024) have 275/75 and 550/150 training/calibration problems and 1,000 and 6,500 disjoint test problems. Training problems have 2–16 draws per route; Omni-MATH and MMLU-Pro test problems have one. Six further pools (AIME (di-zhang-fdu, 2024), APPS (Hendrycks et al., 2021), BigCodeBench (Zhuo et al., 2025), CodeContests (Li et al., 2022), SuperGPQA and BIG-Bench Extra Hard) are split at random into training, calibration and test problems (Appendix A). Table 2 gives each route's accuracy and cost on the three main sets.

Prices. On the Omni-MATH and MMLU-Pro test sets, realized cost is the cost billed for each call; everywhere else (all other pools, and the training problems), it is the call's realized input and output tokens at the effective billed rates. Predictions are priced at each model's effective billed rate, a least-squares fit of billed cost on input and output tokens (USD per million output tokens: 0.13 for gpt-oss-20b, 0.08 for pinned deepseek-v4-flash, 0.26 for gpt-oss-120b). These differ from the list prices by up to 2.3×. The rates are fitted on the test calls; fitting them on training calls only (deepseek-v4-flash: identical; gpt-oss: up to 15% different) moves our saving relative to median pricing by -0.9 to $+0.6$ points on the three main test sets (manifest split), and changes the sign of no comparison in Table 1. SuperGPQA and BIG-Bench Extra Hard were collected later, when StreamLake billed deepseek-v4-flash at about twice this rate; at the rates billed in their own training calls our saving there is 4.9 and 1.1 points higher (manifest split).

Evaluation. All estimators are fitted on training problems only and evaluated on the test sets, with the same success predictions. Each V on a dense grid gives a deterministic policy and an observed (cost, accuracy) point. Comparing two cost estimators a, b, we interpolate each one's cost at 12 accuracies spanning the interior 90% of the range both reach and report $1 - \exp(\overline{\log C_a/C_b})$, the cost saved by a at matched accuracy. Which problems are used for training matters at these sizes, so every comparison of estimators is averaged over ten *training sets*: each reshuffles a pool's training and calibration problems into new splits of the same sizes, keeps the test set, and refits every readout and estimator. We report the mean over training sets. Intervals are percentiles of paired problem bootstraps (300 resamples per training set, keeping each problem's routes together and using the same resamples for both arms), pooled over the ten training sets, so they reflect both the training set and the test problems. Restricted and extended route pools, non-reasoning routes, deployable policies, per-query budgets, transfer to new benchmarks, the text-embedding baselines and mixed traffic are averaged over the same ten training sets. So are the other prefills, the length predictors, the noise ceilings and the variance decomposition of Section 6. The remaining analyses (held-out draws, rescaled prices, the within-test variance decomposition and length drivers of Section 6, fine-tuned encoders on Omni-MATH and MMLU-Pro, parts of the transfer analysis, the prefill router's success pipeline, the perfect-judge cascade and the software-engineering pool) are run on one training set, the split fixed in the manifests, compare arms within it, and are marked "manifest split" where they appear. As an upper bound, an oracle arm prices each problem's realized output length at the billed rates; its saving over median pricing is the headroom, and our share of it the capture. To check that an operating point can be chosen in advance, we also select the cheapest V reaching each accuracy target on calibration and apply it once.

Cost saved by ours vs.	LCB (n=341)	Omni-MATH (n=1,000)	MMLU-Pro (n=6,500)
Training-median length	27.8 [13.9, 35.6]	31.5 [23.9, 37.7]	23.4 [17.1, 29.0]
Training-mean length	27.8 [19.5, 34.6]	34.3 [26.3, 40.1]	25.8 [20.4, 30.6]
Prompt-feature GBM	20.7 [10.1, 29.0]	25.8 [15.9, 35.0]	20.0 [13.9, 25.9]
MixLLM-style embeddings	16.2 [7.9, 22.6]	24.7 [15.9, 31.7]	15.5 [10.6, 20.5]
ZeroRouter (reimplemented)	19.3 [7.7, 28.2]	14.2 [−4.7, 39.3]	18.4 [5.9, 24.2]
Headroom: oracle vs. median	40.1 [29.0, 46.9]	51.9 [44.3, 56.8]	58.0 [53.9, 60.8]

Table 1: **Cost saved at matched accuracy, billed costs.** Cost saved (%) by our prefill cost readouts relative to each estimator, mean over ten training sets with 95% paired bootstrap intervals pooled over training sets; same success predictions for every arm except ZeroRouter, which uses its own. All intervals exclude zero except ZeroRouter's on Omni-MATH, whose calibration-chosen configuration varies between training sets (ours is ahead in 7 of 10). Last row: cost saved over median pricing by perfect per-query cost knowledge (each problem priced at its realized output), with the same success predictions; our readouts capture 69%, 61% and 40% of it. Test sets as in Fig. 2. Prompt-feature GBM: gradient boosting on prompt statistics, as in latency-aware routing (Patel et al., 2026). MixLLM-style: jina-embeddings 137M with the MixLLM ensemble head (LCB, code-aware variant). ZeroRouter: item-response latent fitted on the five routes, 4B features, its own success model and bin pricing, configuration chosen on calibration.

	LCB		Omni-MATH		MMLU-Pro	
Route	Acc.	Cost	Acc.	Cost	Acc.	Cost
gpt-oss-20b, low	51.8	0.13	41.9	0.21	62.1	0.05
gpt-oss-20b, medium	68.2	0.65	57.5	1.20	69.4	0.18
deepseek-v4-flash	89.3	0.90	71.3	1.66	86.2	0.33
gpt-oss-120b, medium	82.1	0.64	60.3	1.12	77.2	0.23
gpt-oss-120b, high	89.5	3.73	66.6	4.57	79.3	0.90

Table 2: **Routes on the test sets.** Accuracy (%) and mean billed cost per call (USD per 1,000 calls). Costs differ 17–30× across routes, and accuracy is not monotone in price: deepseek-v4-flash is the most accurate route or within 0.2 points of it at 24–37% of the cost of gpt-oss-120b at high effort (Section 11).

4 Results

Cost readouts beat every estimator. At matched accuracy, prefill cost readouts spend 27.8% less than median-length pricing on LCB, 31.5% less on Omni-MATH and 23.4% less on MMLU-Pro (Table 1). That is 69%, 61% and 40% of the headroom, the saving perfect per-query cost knowledge would bring (40–58%). Every external estimator loses: mean length by 26–34 points, a prompt-feature model by 20–26, MixLLM-style embeddings by 16–25 and a full ZeroRouter reimplementation by 14–19, all with intervals excluding zero except ZeroRouter on Omni-MATH; a text-embedding model of the prefill's own size loses by 12–23 under every readout (Table 9). Within single training sets, ours is significantly ahead of mean length, prompt features, MixLLM-style embeddings and ZeroRouter in all ten training sets on all three test sets, except ZeroRouter in 3 of the 10 Omni-MATH training sets. Choosing V in advance costs little: policies selected on calibration spend 1.0–3.3% more than our own test frontier at the accuracy they reach on the six pools other than APPS, BigCodeBench and CodeContests (hollow circles in Fig. 2; Section 11).

Pool	Domain	Test n	Headroom	Ours vs. median	Capture	Ours vs. cost-from-success
LiveCodeBench	code	341	40.1	27.8 [13.9, 35.6]	.69	3.2
Omni-MATH	math	1,000	51.9	31.5 [23.9, 37.7]	.61	−3.0
MMLU-Pro	knowledge	6,500	58.0	23.4 [17.1, 29.0]	.40	**18.4**
AIME 1983–2024	math	281	29.2	14.9 [10.8, 20.1]	.51	4.3
APPS	code	377	34.4	16.8 [4.2, 30.4]	.49	−2.2
BigCodeBench	code	421	10.0	0.4 [−9.5, 9.1]	.04	2.4
CodeContests	code	266	35.8	9.4 [0.4, 19.8]	.26	−0.8
SuperGPQA	knowledge	300	60.6	33.0 [14.6, 45.7]	.54	15.3
BIG-Bench Extra Hard	reasoning	300	38.1	23.5 [9.4, 35.2]	.62	**22.6**

Table 3: **Headroom and capture on every pool.** Cost saved (%) at matched accuracy relative to median-length pricing: by perfect per-query cost knowledge (headroom) and by our readouts (95% paired bootstrap intervals pooled over training sets); capture is their ratio. Means over ten training sets. Last column: cost saved by our dedicated cost readout relative to our ablation that prices from the success readouts alone; bold: interval excludes zero. Billed costs, deepseek-v4-flash pinned, same success predictions in every column.

Other pools. On the remaining six pools (Table 3) our readouts save 14.9% [10.8, 20.1] on AIME, 16.8% [4.2, 30.4] on APPS, 33.0% [14.6, 45.7] on SuperGPQA and 23.5% [9.4, 35.2] on BIG-Bench Extra Hard relative to median pricing; Section 8 treats BigCodeBench, where no estimator saves significantly, and CodeContests, where our saving barely excludes zero. No external estimator is significantly cheaper than ours on any pool (Table 11). Across training sets our saving varies with an s.d. of 0.9–4.4 points per pool; random re-splits that also reshuffle the test problems give the same picture (Table 18).

5 How much is per-query cost worth?

A cost estimator can be no better than knowing each query's realized cost. We therefore measure, on every pool, the *headroom*: the cost saved at matched accuracy when each test problem is priced at its realized output length instead of the training median, with the same success predictions. Table 3 reports it with our saving and the share of it we capture, on nine pools under one protocol (five routes, deepseek-v4-flash pinned, billed rates, test splits).

Headroom is large wherever reasoning lengths vary. On eight of nine pools perfect per-query cost knowledge would save 29–61%, and on seven of them our readouts capture two fifths to over two thirds of it (.40–.69). The exceptions are informative. BigCodeBench has little to capture (10.0%), and CodeContests has headroom that our readouts barely reach (capture .26); Section 8 shows that no estimator reaches it.

The headroom is an upper bound. Pricing each problem at its realized cost also knows the sampling noise of the draws it is scored on, which no predictor of expected cost can. Where test problems have several draws per route, we split each problem's draws at random into a half that prices and a half that is scored (20 splits). On the manifest split, pricing at the held-out half's mean length, a target an expected-cost predictor could reach, saves 41.2% on LiveCodeBench against 45.9% for the realized-cost oracle on the same halves; the gap is larger where draws are few (AIME 20.1 against 30.1, Omni-MATH-500 23.6 against 35.9, CodeContests 23.4 against 38.9) and small on SuperGPQA (52.7 against 57.1). With one or two draws per half the held-out oracle is itself noisy, so the achievable headroom lies between the two; on LiveCodeBench, where it is tightest, our readouts capture .81 of the held-out bound against .72 of the realized one. The capture figures in Table 3 are therefore conservative.

Length, not only price gaps, makes it pay. To separate the two sources of headroom, we rescale every route's billed price toward or away from the cheapest route's while keeping realized lengths fixed. On the manifest split, with all routes at the same per-token price, our readouts still save 13.3% (LCB), 14.3% (Omni-MATH) and 36.3% (MMLU-Pro) relative to median pricing: per-query length alone is worth that much. On LCB and Omni-MATH the saving grows with the price gap and saturates near the billed gap (3.1×); on MMLU-Pro it is largest at flat prices (36.3%, against 28.8% at billed prices on the same split),

because there length differs between questions rather than between routes. Widening the gaps beyond the billed ones shrinks the headroom (Fig. 5 in the appendix): at a 29× spread between the most and least expensive route it is 7–49% across pools, at 87× 2–42%, and our saving falls with it (1–25% at 87×). When prices differ by two orders of magnitude, choosing the model dominates and per-query length matters little, as for the software-engineering agents of Section 9.

Which routes the saving needs. We refit the router on every subset of the five routes that leaves out one route, and on every pair (Appendix J; ten training sets). Choosing effort within one model is worth little: with only gpt-oss-20b's two efforts ours saves 20.2% (LCB), 16.8% (Omni-MATH) and 9.0% (MMLU-Pro) relative to median pricing, with only gpt-oss-120b's 0.2%, 7.0% and 6.0%. Leaving out the cheapest route, gpt-oss-20b at low effort, removes most of the saving (LCB 27.8% to 6.5%, Omni-MATH 31.5% to 16.2%, AIME 14.9% to −0.2%, SuperGPQA 33.0% to 0.7%, BIG-Bench Extra Hard 23.5% to 3.8%); leaving out deepseek-v4-flash removes less (to 22.4%, 15.2%, 7.0%, 15.4% and 26.2%; MMLU-Pro 23.4% to 17.0%), and leaving out either gpt-oss-120b effort or gpt-oss-20b at medium effort changes the saving by at most 2.1 points on LCB, Omni-MATH, MMLU-Pro, AIME, SuperGPQA and BIG-Bench Extra Hard.[1] Among pairs, the saving lives in the cheap route paired with a strong route whose length varies widely: gpt-oss-20b at low effort with deepseek-v4-flash alone saves 33.8% on Omni-MATH, more than the full pool, and 27.0% on MMLU-Pro; with gpt-oss-120b at high effort it saves 32.1% on LCB. deepseek-v4-flash has the widest length spread of any route on all nine pools (s.d. of log length 1.72 on LCB and Omni-MATH, against at most 1.40 and 1.47 for the other routes), and on seven pools its spread of cost per call exceeds the spread of prices between routes; our readout reads its length no better than the other routes' (Table 15), and on the knowledge pools worse (Table 12). Per-query cost decides, query by query, whether a strong route whose cost varies widely is worth calling instead of a cheap one.

Reasoning makes per-query cost worth predicting. To test whether this is a property of reasoning models, we collected five non-reasoning routes on the same problems: deepseek-v4-flash with thinking disabled (same provider and sampling), Llama-3.1-8B (Grattafiori et al., 2024), Qwen3-30B-A3B-Instruct, Llama-3.3-70B and Qwen3-235B-A22B-Instruct, each pinned to one provider, one draw per problem, with output capped at 16k tokens. Readouts are fitted exactly as for the reasoning routes. Table 4 compares the two pools on the common test problems. Headroom is 1.3–2.3× larger with reasoning routes, and our saving is 2.3× (LCB) to about 30× (MMLU-Pro) larger; on MMLU-Pro the non-reasoning saving is not distinguishable from zero in any training set. Non-reasoning lengths do vary (s.d. of log length .53–1.73 per route), but much of the variation is degenerate output that the prompt predicts poorly: Llama-3.1-8B falls into repetition until the 16k cap on 9–12% of math problems, and Qwen3-30B-A3B-Instruct, asked for code only, reasons inside code comments on hard LiveCodeBench problems (Appendix F). Adding the non-reasoning routes to the reasoning pool lowers our saving (14.8% on Omni-MATH, 7.6% on MMLU-Pro, 19.2% on LCB). The same model with thinking on and off shows the trade directly: deepseek-v4-flash with thinking gains 2.8 (MMLU-Pro), 14.6 (Omni-MATH) and 22.1 (LCB) points of accuracy for 13×, 11× and 33× the output, and the problems that are long with thinking are long without it (correlation of log length .66–.76).

6 What the cost readout captures

Length is mostly a property of the problem. Fitted on the training problems, the prefill explains much of each route's log output length on the test problems (R^2 .71–.75 on LCB, .66–.73 on Omni-MATH, .41–.51 on AIME and .26–.62 on MMLU-Pro; ten training sets, Table 12). A decomposition by 5-fold cross-validation within each pool's test problems (manifest split) shows that on LCB, Omni-MATH and AIME most of this is shared with difficulty: a problem's empirical solve rate across routes alone reaches R^2 .48–.65, and the prefill misses at most .07 of what difficulty explains (.10–.22 on AIME). Within a problem, failed and solved draws are about equally long (×0.83–1.16), with one exception: deepseek-v4-flash writes 2.5× (Omni-MATH) and 1.9× (AIME) longer when it fails on hard math.

[1] Without gpt-oss-20b at low effort on MMLU-Pro, deepseek-v4-flash has the lowest median cost per call, the median rule sends almost every question to it, and its frontier no longer overlaps ours, so no saving is defined.

	Headroom		Ours vs. median		Capture	
Test set	reasoning	non-reas.	reasoning	non-reas.	reasoning	non-reas.
LiveCodeBench (341)	40.1	21.4	27.8	12.1 (5.4)	.69	.53
Omni-MATH (1,000)	53.1	40.5	31.5	6.8 (1.6)	.59	.17
MMLU-Pro (2,000)	59.8	26.0	25.8	0.8 (2.2)	.43	.03

Table 4: **Reasoning versus non-reasoning routes on the same problems.** Headroom (perfect per-query cost knowledge vs. median pricing), our saving relative to median pricing (in parentheses: s.d. over training sets) and capture, for the five reasoning routes and for five non-reasoning routes, on the test problems where every route of both pools has a valid answer (MMLU-Pro: a 2,000-problem subset of the test set). Means over ten training sets; billed costs; readouts fitted on the same training problems with the same recipes. Here the oracle prices each problem at its billed cost, so on Omni-MATH the reasoning headroom differs slightly from Table 3 (51.9), which prices realized output at the fitted rates. The non-reasoning saving excludes zero in every training set on LCB and Omni-MATH and in none on MMLU-Pro. Qwen3-235B-A22B-Instruct has training answers for 241 of 275 Omni-MATH and 485 of 550 MMLU-Pro problems (rate limits).

What the data fix, independent of any predictor. Three quantities need no predictor (Appendix H). First, the share of a route's log-length variance that is fixed by the problem rather than by sampling (a noise ceiling from repeated draws: the R^2 no predictor of the prompt can exceed) is .71–.99 for every reasoning route on every pool where we have repeated draws. Second, other routes' realized lengths on the same problem explain .42–.96 of a route's log length (every route except gpt-oss-20b at low effort on BIG-Bench Extra Hard), so much of what fixes a route's length is a property of the question that models agree on. Third, the variance of log length (excluding route means) splits into a shared problem level (42–84%), a problem-by-route interaction (8–29%) and run-to-run noise (8–29%). The interaction is small, but it carries as much routing value as the level or more: an oracle that knows each test problem's route-specific part only saves 24.8%, 41.4% and 43.7% relative to median pricing on LCB, Omni-MATH and MMLU-Pro, against 25.5%, 23.3% and 25.4% for one that knows its shared level only (headroom 40.1%, 51.9% and 58.0%; ten training sets). On every pool the route-specific part is worth at least as much as the level, within a point.

Where our readout falls short. Our readout is the best pre-generation length predictor we have on every route of every pool, ahead of pricing from success, ZeroRouter's latent, prompt features, MixLLM-style and nearest-neighbour readouts of a text encoder, three readouts of Qwen3-Embedding-4B and, on LCB, the fine-tuned Qwen3-0.6B (Table 12). Its R^2 is a lower bound on how legible length is, and the noise ceiling an upper bound. The two are close on LCB, Omni-MATH and BIG-Bench Extra Hard (e.g. LCB .71–.75 against .98–.99), and far apart on the knowledge pools, for deepseek-v4-flash above all: on MMLU-Pro and SuperGPQA we read .26 and .22 of its log length, other routes' lengths explain .62 and .46, and the ceiling is .92 and .96. Its length there is fixed by the question and largely shared with other models, but the prefill misses it. Route differences are what predictors read worst: across route pairs, our R^2 for the log length ratio is .06–.47 per pool, against a ceiling of .48–.85, and this differential R^2 tracks the share of a pair's headroom we capture (Spearman .49 over the 75 pairs with headroom above 5 points) better than per-route R^2 does (.18). On LCB (manifest split), the prefill reads the level (R^2 .75) far better than the differences (.41), and pricing with the predicted level alone keeps 93% of the full readout's saving.

Where length tracks work rather than difficulty. On the 300-problem test split of the MMLU-Pro pool (manifest split), difficulty explains log length only in part (test R^2 .18–.37), subject labels about as much (.17–.25), and the two together .34–.44, against .41–.66 for the prefill. Engineering and law questions are both long (6.4k and 4.1k output tokens on average) and hard. Pricing from difficulty and subject together saves 32.6%, against 43.9% for the prefill (headroom 61.5%): the prefill reads how much work a question requires, beyond its difficulty and topic.

The success readouts carry more than difficulty. Even where pricing from success ties the dedicated readout, it does not price by difficulty alone. On the manifest split, a problem's true solve rate over the five

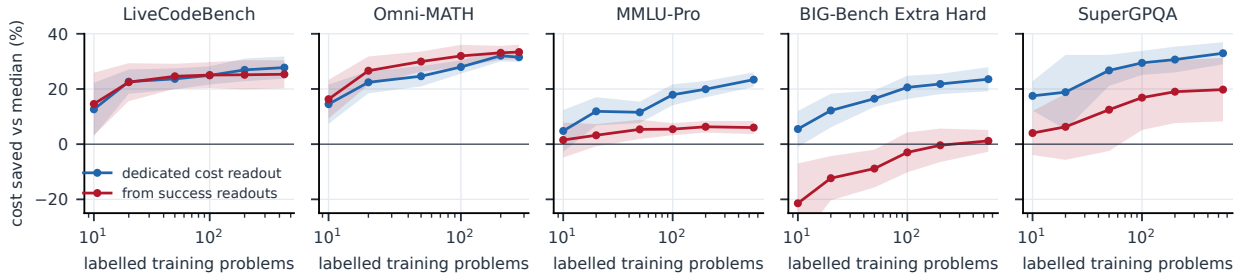


Figure 3: **Length labels needed.** Cost saved relative to median pricing when the cost readout is fitted on n labelled training problems: a dedicated readout on the prefill (blue) versus pricing from the success readouts (red), which needs length labels only to map success logits to length. Mean over ten training sets with two random subsets each; bands ±1 s.d. over training sets. Where length tracks difficulty (LiveCodeBench, Omni-MATH), pricing from success is as good from 20 problems on; where it tracks the work a question requires (MMLU-Pro, BIG-Bench Extra Hard), it stays below 7% at every n.

routes (with its square) is a weak price even in-domain: it saves 21.5% on LCB against 34.4% for pricing from the success logits, and nothing on the 500-problem Omni-MATH pool (−4.1% against 18.0%), because the logits carry length cues beyond the solve rate. The part of the predicted length that the true solve rates do not explain tracks the same part of the realized length (correlation .56–.92 across six pools; Appendix G). We read it as the work a problem requires, which difficulty proxies only in part.

Two further heterogeneous pools. We chose SuperGPQA (M-A-P Team, 2025) and BIG-Bench Extra Hard (Kazemi et al., 2025) as further pools because, on a cheap screen (one gpt-oss-20b-low draw per problem), length predicted directly from the prefill beat length predicted from a success readout by a margin as large as MMLU-Pro's (R^2 gap .61 and .40, against .66 on MMLU-Pro and .10 on Omni-MATH). On their full pools the dedicated readout again beats pricing from success on BIG-Bench Extra Hard, by 22.6% [12.3, 31.9], where pricing from success saves only 1.2% over the median; on SuperGPQA the margin is smaller and its interval just includes zero (15.3% [−0.6, 32.3]; ahead in 7 of 10 training sets; Table 3), so the screen overstated the separation there. On BIG-Bench Extra Hard, text-only estimators read length as well as the prefill: MixLLM-style embeddings and prompt features save 24.4% and 21.9% relative to median pricing (ours −1.3 [−5.1, 2.7] and 2.1 [−3.8, 7.0] relative to them). Its 23 tasks have very different formats, and the task is visible in the text. On SuperGPQA they do not (ours saves 15.7% [0.1, 27.5] relative to MixLLM-style and 19.1% [3.1, 32.2] relative to prompt features).

How many length labels are needed. Given success readouts, where length tracks difficulty a cost readout needs few length labels (Fig. 3). Pricing from the success logits reaches 22.5% (LCB) and 26.6% (Omni-MATH) with 20 labelled training problems, against 22.6% and 22.4% for a dedicated readout. On MMLU-Pro it plateaus at about 6%, while the dedicated readout climbs to 17.9% at 100 problems and 23.4% at all 550. One sampled output per training problem instead of all of them gives up 1–3 points (manifest split).

7 How cheaply can a router get it?

Where length tracks difficulty, pricing from the success readouts ties the dedicated cost readout when both are fitted on a benchmark's full training set (Table 3). A deployed router rarely has that: it has few labels, it adds models, and it meets benchmarks it was not trained on. We test the cost estimators in all three settings, with every estimator given the same success predictions unless stated, and the same protocol as before, and finally route one mixture of all nine benchmarks.

Few labels. Table 5 refits every cost estimator on 20 randomly chosen training problems (two subsets in each of the ten training sets). Our readout keeps 51–81% of its full-data saving on every pool (LCB

Cost estimator, 20 training problems	LCB	Omni-MATH	MMLU-Pro	SuperGPQA	BBEH
Ours (prefill readout)	**22.6**	**22.4**	11.9	18.9	12.2
Cost from success (our ablation)	22.5	26.6	3.2	6.3	−12.3
Prompt-feature GBM	−1.0	−2.3	2.2	8.1	0.9
MixLLM-style embeddings	−2.9	1.9	−0.5	5.0	−5.4
CARROT-style kNN	1.7	3.7	2.3	9.5	3.9
ZeroRouter pricing	1.7	5.4	0.2	−2.7	−1.0
Ours, all training problems	27.8	31.5	23.4	33.0	23.5

Table 5: **Few labels.** Cost saved (%) relative to median pricing (from the full training set) when each cost estimator is fitted on 20 random training problems; mean over ten training sets with two subsets each; every arm routes with the same success readouts. ZeroRouter pricing: its item-response latent fitted on the 20 problems, read from the prefill, with its bin lookup; kNN neighbours and ZeroRouter's dimension and bin count chosen on the calibration split. Bold: ours exceeds every external estimator with paired intervals (pooled over training sets) excluding zero (LCB and Omni-MATH from n=20 except Omni-MATH at n=50, MMLU-Pro from n=100, SuperGPQA from n=200); on BIG-Bench Extra Hard the text estimators stay within noise of ours at every n.

22.6%, Omni-MATH 22.4%, MMLU-Pro 11.9%, SuperGPQA 18.9%, BIG-Bench Extra Hard 12.2%), while the external estimators save less than 10% on every pool. Their deficit narrows with data but stays large on most pools: at 100 problems ours saves 17.9% on MMLU-Pro against at most 6.5% for any external estimator, with every paired interval excluding zero. The exception is BIG-Bench Extra Hard, where the task format is visible in the text: the text estimators reach 15–17% at 100 problems and tie ours at full data; below that ours leads on average (by 16.5 points over MixLLM-style embeddings at 50), with intervals that include zero. Pricing from the success readouts matches or exceeds ours on LCB and Omni-MATH from 20 problems on, as in Fig. 3, and is far behind ours on the two heterogeneous pools.

New models. To add a model, a router needs its success and cost readouts. We hold out one model at a time from the pool (both efforts of gpt-oss-20b, both efforts of gpt-oss-120b, or deepseek-v4-flash, so that no sibling effort of the new model remains) and fit its readouts from k labelled problems while every other route keeps its full readouts. Our onboarding fits two parameters per readout: an offset to the other routes' predicted log length, and a logistic map from their mean success logit. ZeroRouter's onboarding, its main use case, fits the new model's ability on the same k anchors in an item-response latent fitted on the remaining routes and fills its per-bin length table from them (Yan et al., 2026); a CARROT-style arm averages the k anchors' outcomes and lengths over nearest neighbours. Table 6 reports the mean over the three held-out models. From 10 examples our router saves 19.2–26.7% on four pools and 9.7% on BIG-Bench Extra Hard, against 3.4–17.4% for ZeroRouter's onboarding. It leads ZeroRouter on every pool on average (by 5.2–13.0 points), but at k=10 every interval includes zero; at k=50 the lead is significant on LCB (9.6 [3.0, 18.7]) and SuperGPQA (12.8 [4.8, 20.8]), and within noise on the other three. The gain concentrates on deepseek-v4-flash, the one model whose family is not otherwise in the pool: onboarding it from 10 problems recovers 13–31 points more than ZeroRouter's onboarding does. ZeroRouter's Fisher-information anchors help both methods about equally; with the same anchors ours ties on LCB and Omni-MATH and leads by 3.5–12.9 points on the other three pools at k=10.

Three further model families. Adding three reasoning models from other families (Qwen3-32B, GLM-4.7-Flash and Nemotron-3-Super-120B) on LCB, Omni-MATH and a 2,700-problem subset of MMLU-Pro leaves the picture unchanged (Appendix I). With all eight routes our readouts save 28.8%, 25.6% and 23.0% relative to median pricing. Without deepseek-v4-flash, in a pool of both gpt-oss-20b efforts and the three new models, they save 29.5% [17.3, 37.2], 12.4% [−0.2, 21.7] and 22.7% [9.9, 33.6], so the gain does not rest on one cheap and accurate route (means over ten training sets; significant on LCB and MMLU-Pro). The new models matter only in that pool; onboarding Nemotron-3-Super-120B there from 10 problems, our router saves 15.4% on LCB and 15.5% on MMLU-Pro, against −35.1% and −6.6% for ZeroRouter's onboarding.

	LCB	Omni-MATH	MMLU-Pro	SuperGPQA	BBEH
Ours, k=10	19.5	22.7	19.2	26.7	9.7
ZeroRouter onboarding, k=10	7.3	17.4	9.8	13.7	3.4
CARROT-style kNN, k=10	5.2	11.4	9.5	13.8	8.3
Median length + base rate, k=10	−13.6	7.6	2.1	9.0	1.8
Ours − ZeroRouter, k=10	12.2 [−5.9, 26.7]	5.2 [−7.9, 18.2]	9.4 [−0.9, 19.3]	13.0 [−1.8, 25.9]	6.3 [−11.5, 29.1]
Ours − ZeroRouter, k=50	9.6 [3.0, 18.7]	5.1 [−5.4, 17.7]	8.1 [−0.5, 16.5]	12.8 [4.8, 20.8]	4.8 [−1.5, 10.5]
All routes on full readouts	27.8	31.5	23.4	33.0	23.5

Table 6: **Onboarding a new model from k examples.** Cost saved (%) relative to the full pool at median pricing, mean over three held-out models (gpt-oss-20b with both efforts, gpt-oss-120b with both efforts, deepseek-v4-flash), ten training sets and four random choices of the k anchors in each (the same anchors in every arm); every other route keeps its full readouts. Differences in points with 95% paired bootstrap intervals over test problems, pooled over training sets. ZeroRouter's dimension and bin count and the kNN neighbour count are chosen on calibration.

New benchmarks. Finally, we move a whole router to a benchmark it has not seen: both readouts are trained on the other pools of the same domain (code, or math and knowledge), with every choice (regularisation, Platt calibration, neighbour counts, latent dimension, bin count) made on those pools' calibration splits, so no label of the target is used. Table 7 compares it with the router trained on the target itself (means over ten training sets, each refitting every pool's readouts). On LCB and AIME the transferred router keeps 94–96% of its in-domain saving (26.0% and 14.3%), and on MMLU-Pro and APPS it exceeds it (26.6% and 31.5%). The prefill router's rule, the same transferred success readouts with median pricing, saves at most 9.0% on any of these and loses money on LCB, so the transferred cost readout carries the result, even though transferred success predictions are weaker (LCB AUC .67 against .83). Ours leads the best external estimator transferred the same way by 28.9 points on LCB and 8.5 on MMLU-Pro (intervals excluding zero in every training set), 7.5 on AIME (in 9 of 10) and 7.8 on APPS (in 5 of 10). Transfer is weaker on SuperGPQA (22.5%), Omni-MATH (4.0%) and BIG-Bench Extra Hard (−3.4%); there ours is ahead of the best transferred external estimator by 2.4–4.9 points on average, but significantly in at most two training sets. On the manifest split, combining both sources of labels is better than either: with ten labelled target problems added to the source data, the cost readout saves 27.9% on MMLU-Pro and 13.1% on AIME (with the target's success readouts), against 8.1% and 4.4% from the ten problems alone, and 24.1% against 14.7% on BIG-Bench Extra Hard with twenty. Pricing from the success readouts transfers unevenly (manifest split). With the target's own success readouts and a map trained on the other pools, it keeps its saving where the target's kind of difficulty resembles its sources' (APPS 17.1%, AIME 12.3%, CodeContests 9.3%) and collapses elsewhere: from 34.4% to −6.2% on LCB, 19.6% to −21.9% on SuperGPQA, 18.0% to 6.5% on Omni-MATH and 5.5% to −20.5% on BIG-Bench Extra Hard. Under the same conditions the dedicated readout saves 30.2%, 17.7%, 15.7% and 16.1% on those four, and ZeroRouter's difficulty-bin pricing transfers as unevenly as pricing from success.

Which estimators lose least. On the manifest split, holding the target's success readouts fixed, we bootstrap over test problems the drop from in-domain to transferred pricing for each estimator, and its difference from our drop. Ours loses significantly less than pricing from success on LCB (37.1 points [26.9, 46.0]), SuperGPQA (31.8 [16.3, 49.9]) and BIG-Bench Extra Hard (13.4 [1.4, 23.9]), and less than ZeroRouter's bin pricing on LCB (21.0 [11.7, 28.5]) and MMLU-Pro (4.0 [1.1, 8.2]); on the other pools the drops are within noise of each other, and on no pool does ours lose significantly more. Our own drop is significant only on the two heterogeneous pools (9.8 and 12.5 points).

Why the cost readout transfers. On the manifest split, training one readout on the target together with the other pools matches the target-only readout within 4.4 points on every pool, with no difference significant, so a single readout can serve all of them. Part of what transfers is shared structure: 24–40% of each route's variance in log length lies between benchmarks, and the routes agree on which benchmarks are long (mean correlation of benchmark means across routes .84). Length R^2 under transfer is close to

Target	Ours, in-domain	Ours, transferred	Prefill-router rule, transferred	Best external, transferred
LiveCodeBench	27.8	26.0	−9.4	−3.0 (ZeroRouter)
APPS	16.8	31.5	9.0	23.7 (ZeroRouter)
MMLU-Pro	23.4	26.6	1.1	18.1 (kNN)
AIME	14.9	14.3	−1.7	6.8 (kNN)
SuperGPQA	33.0	22.5	−11.2	17.6 (MixLLM)
BIG-Bench Extra Hard	23.5	−3.4	−21.0	−6.5 (ZeroRouter)
Omni-MATH (500-problem pool)	17.8	4.0	−23.4	1.7 (MixLLM)

Table 7: **A router moved to a new benchmark.** Cost saved (%) relative to the target's own median rule, at matched accuracy on its test set; means over ten training sets (every pool's splits reshuffled with the same seed). Transferred: success and cost readouts trained on the other pools of the same domain (coding: LCB, APPS, BigCodeBench, CodeContests; math and knowledge: Omni-MATH, MMLU-Pro, AIME, SuperGPQA, BIG-Bench Extra Hard), with every choice made on their calibration splits. Prefill-router rule: the transferred success readouts with median pricing. External estimators transferred the same way, each with its own success model (MixLLM-style: logistic readout on the text embedding; kNN; ZeroRouter: its latent and bins). Ours exceeds every other arm with paired intervals excluding zero in every training set on LCB and MMLU-Pro and in 9 of 10 on AIME. Omni-MATH uses its 500-problem pool (test 150), whose prefill matches the other pools'. BigCodeBench and CodeContests, where no in-domain estimator saves significantly, are omitted: there the transferred success readouts are badly miscalibrated (BigCodeBench log loss 1.17 against .67 in-domain) and the apparent gains are not robust.

in-domain on LCB, APPS, AIME, Omni-MATH and SuperGPQA but falls sharply on MMLU-Pro (.02–.46 against .29–.63), where routing transfers fully nonetheless: routing needs the ordering of problems within a route and the contrast between routes, not calibrated absolute lengths. What does not transfer is the map from difficulty to length. Matching the problems of two benchmarks by solve rate (per route, five bins), their lengths still differ by 1.14 log units on average (×3.1 over 470 cells). Our cost readout, trained on neither benchmark, predicts these gaps (correlation .95, 90% of the squared gap), pricing from success 30%, and any price from difficulty alone none, by construction. The curve from solve rate to length also differs in shape: it is flat on BigCodeBench, and without BigCodeBench among LCB's sources a linear map from the mean success logit to length transfers to LCB (12.5% to 32.5%; in-domain 34.7%). Difficulty is a benchmark-specific proxy for work; the prefill readout reads cues of the work itself, which carry over when the sources contain the target's kinds of work.

Mixed traffic. A deployed router sees queries from many sources, and recent evaluations find that much of what quality routers gain can be had by knowing a query's dataset or task type (Li et al., 2026a; Lee, 2026). A cost router could likewise get most of its saving by segmenting traffic. We route one mixture of all nine benchmarks with the same five routes: training and calibration problems are the union of each pool's splits, the test set has 150 problems from each pool (1,350), and both readouts are refitted on the mixture (Omni-MATH enters as its 500-problem pool, whose prefill matches the others'). Table 8 compares the cost readout with routers that are told each query's benchmark. Pricing each route at its median length within the query's benchmark saves 12.2% over a single median per route; adding difficulty within the benchmark, by fitting pricing from the success readouts or ten difficulty bins separately per benchmark, raises this to 24.4% and 24.1%. Our single readout, which is not told the benchmark, saves 31.8%: 22.4% [16.5, 27.6] beyond benchmark identity, and 9.9% [5.2, 14.2] beyond benchmark identity and difficulty together, with intervals excluding zero in all ten training sets. Pricing from success fitted on the mixture without the benchmark saves only 11.7%. The readout recovers the benchmark-level structure without being told it and adds query-level cost information that neither the benchmark nor difficulty carries. Perfect per-query cost knowledge would save 38.9% beyond benchmark-aware median pricing, of which our readout captures .58.

Router prices each route by	Cost saved relative to one median	Cost saved relative to median per benchmark	Ours vs. it
One median length (the prefill router's rule)	–	−13.9	31.8 [26.1, 36.7]
Median length per benchmark	12.2	–	22.4 [16.5, 27.6]
Mean length per benchmark	13.5	1.5	21.2 [15.9, 26.0]
Success readouts, fitted per benchmark	24.4	13.8	9.9 [5.2, 14.2]
Difficulty bins, per benchmark	24.1	13.6	10.1 [4.4, 15.1]
Success readouts, fitted on the mixture	11.7	−0.5	22.8 [17.8, 27.8]
Ours (one readout, benchmark not given)	**31.8**	**22.4**	–
Realized cost (headroom)	46.4	38.9	

Table 8: **One router for mixed traffic.** All nine benchmarks in one pool (150 test problems each), the same five routes and the same success readouts in every row, fitted on the mixture. Cost saved (%) at matched accuracy, mean over ten training sets; 95% paired bootstrap intervals (resampled within each benchmark) pooled over training sets. Every "ours vs. it" interval excludes zero in all ten training sets. Per-benchmark arms are told each query's benchmark; ours is not.

8 When it works and when it does not

Per-query cost pays under two conditions: (a) output length varies within a route by more than prices differ between routes, which creates headroom, and (b) an estimator can read that variation from the prompt, which lets it capture the headroom. For every reasoning route here the variation is mostly fixed by the problem rather than by sampling (noise ceiling .71–.99; Section 6), so (b) is a property of the problem and the estimator together. Each of our two failures violates one condition, and neither is specific to our estimator (Fig. 4). Neither condition reduces to a law in length and price statistics alone: across 138 route subsets of the nine pools, headroom is not predicted out of sample by the price spread between routes, the variance shares of Section 6, or their ratio (leave-one-pool-out R^2 at most .25, for the price spread alone; Appendix H), because it also depends on which routes solve which problems.

Little headroom: BigCodeBench. BigCodeBench tasks are library-usage functions with short, similar solutions; perfect cost knowledge saves only 10.0% [−1.1, 20.1]. No estimator's saving relative to median pricing has an interval excluding zero: ours 0.4 [−9.5, 9.1], ZeroRouter 8.5 [−13.0, 26.5], MixLLM-style −1.2, cost-from-success −2.0, prompt features −1.1. Here length is the least tied to difficulty of any pool (the prefill-only share of length variance is largest on BigCodeBench), but there is little to gain from reading it.

Headroom no estimator reads: CodeContests. CodeContests has as much headroom as APPS (35.8% [26.5, 43.7]), but no estimator captures more than about a quarter of it: ours saves 9.4% [0.4, 19.8] and difficulty bins 7.7% [0.1, 18.1], barely excluding zero; ZeroRouter (9.9% [−4.3, 23.6]), pricing from success (9.9% [−0.4, 21.4]) and the rest are within noise of zero. Length again tracks difficulty, but difficulty itself is hard to read from these prompts: the prefill misses .11–.18 of the length R^2 that empirical difficulty explains, against at most .07 on LCB and Omni-MATH (manifest split; Section 6). The lengths themselves are fixed by the problem (noise ceiling .92–.96) and shared across routes (other routes' lengths explain .67–.88), against our R^2 of .35–.52, so the shortfall is in reading them from these prompts, for every estimator we tried.

Non-reasoning routes. Non-reasoning routes combine both failures (Table 4): less headroom, and on Omni-MATH and MMLU-Pro a capture of .17 and .03. No other estimator does better. Fitting every estimator of Tables 9 and 11 on the non-reasoning routes (ten training sets), none of mean length, prompt features, MixLLM-style or nearest-neighbour embeddings, ZeroRouter's bins and the three Qwen3-Embedding-4B readouts saves significantly relative to median pricing on either pool (at most 1.9%); on Omni-MATH ours is ahead of each by 4.8–12.2 points with pricing from success tied, and on MMLU-Pro every estimator, ours included, is within noise of median pricing. Most of their length is nonetheless readable: on Omni-MATH our readout explains .55–.75 of four routes' log length, about what the other non-reasoning routes' lengths on the same problem explain (.48–.70). The exception is Llama-3.1-8B, whose loops on math problems make its mean output 4–6× its median: no estimator reads its length (best R^2 .10–.22 across the three pools) and

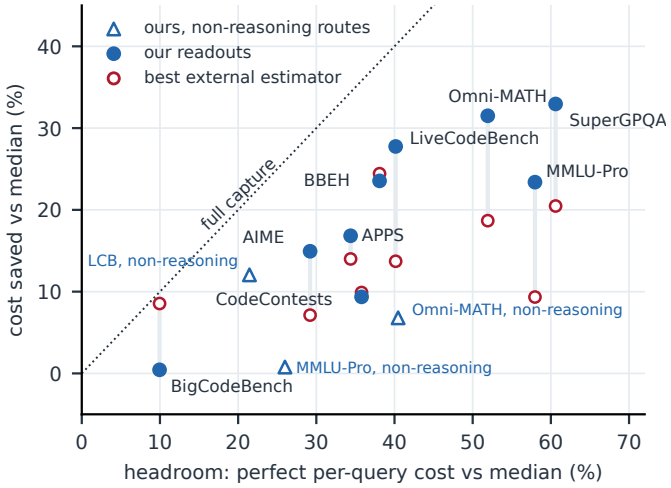


Figure 4: **Headroom and capture.** Per pool: the saving of perfect per-query cost knowledge over median pricing (x) against the saving of our readouts (filled) and of the best external estimator on that pool (hollow; mean length, prompt features, MixLLM-style embeddings or ZeroRouter). Triangles: our readouts with five non-reasoning routes on the same LiveCodeBench, Omni-MATH and MMLU-Pro test problems (Table 4). Billed costs, test splits, deepseek-v4-flash pinned; every point is a mean over ten training sets. Where headroom is large and length is legible, the readouts capture half or more of it; on BigCodeBench there is little to capture, on CodeContests no estimator captures it, and non-reasoning routes leave less headroom, of which no estimator captures more than about a sixth on Omni-MATH and MMLU-Pro.

neither do other models' lengths (.14–.40). What can be read is worth little here, and the most erratic route cannot be read at all. On LCB the readouts capture .53 of a smaller headroom, with pricing from success and the Qwen3-Embedding-4B ridge readout close behind (11.3% and 10.4% against 12.1%).

9 Software engineering

We study software engineering in two regimes: one-shot patch generation, where per-query cost pays as on the coding pools, and agents, where the price gaps between models dominate.

One-shot patches. On 1,432 SWE-Smith instances (Yang et al., 2025b), each of the five routes writes one patch (deepseek-v4-flash pinned as elsewhere), labelled by executing the repository's tests in a sandbox. Pass rates are low and close together (25–37%), but which instances are solvable is legible from the issue (success AUC .80–.83 per route on the 389 test instances where every route returned a patch). Output length varies 5–28× between a route's 10th- and 90th-percentile instances and is only modestly predictable (log-length R^2 .09–.37). On the manifest split, per-query cost knowledge is worth 25.2% [15.0, 31.1] over median pricing, and our readouts save 14.5% [5.8, 19.8], a capture of .58. They beat mean length by 7.8% [3.5, 13.2], pricing from success by 9.2% [2.2, 13.9], MixLLM-style embeddings by 6.8% [2.5, 10.6] and prompt features by 18.2% [8.1, 27.6]; difficulty bins (2.9% [−0.3, 5.7]) and ZeroRouter (1.7% [−4.7, 9.0]) are within noise. One-shot patch generation thus behaves like the coding pools.

Agents. For agents, the price gaps between models dominate. In public SWE-rebench trajectories (Badertdinov et al., 2025) (111 tasks run five times by each of 13 models), 74% of the variance of log cost lies between models. Within a model, per-task cost is stable across runs (intraclass correlation .69–.92), but knowing it pays only when prices are close: it is worth 8.5% [−5.0, 17.8] among all 13 models, whose prices per run span about 190×, and 25.7% [6.3, 40.6] among the seven open-weight models, which span about 10×. From the issue text alone it is only partly legible (R^2 .29 on 3,387 tasks with about 20 SWE-agent runs each (Nebius, 2024); Appendix M).

	Ours vs. embedding cost readout			Ours vs.	Success AUC	
Pool	Ridge	MixLLM	kNN	text router	text	ours
LiveCodeBench	**16.9**	**19.5**	**23.1**	**12.0**	.75	.83
Omni-MATH	**15.8**	**18.8**	**19.6**	**18.5**	.73	.83
MMLU-Pro	**15.8**	**13.3**	**12.0**	**12.6**	.64	.72
AIME	**9.7**	**10.1**	**12.2**	**6.8**	.65	.78
APPS	6.5	11.7	9.0	5.0	.75	.84
BigCodeBench	1.2	2.0	0.8	4.3	.61	.64
CodeContests	1.3	3.0	5.7	−4.7	.65	.74
SuperGPQA	7.0	10.3	**13.3**	7.0	.54	.57
BIG-Bench Extra Hard	5.4	4.1	4.6	8.4	.76	.79

Table 9: **A text-embedding model of the prefill's size.** Cost saved (%) by our prefill router relative to routers that read cost from Qwen3-Embedding-4B with our ridge recipe, a MixLLM-style ensemble or CARROT-style kNN (all with our success readouts), and relative to a fully text-based router (success and cost both from the embedding). Mean over ten training sets; bold: 95% paired bootstrap interval, pooled over training sets, excludes zero. Success AUC: mean over routes on the test set.

10 Ablations

Is difficulty enough to price? Two ablations price each route from our own success readouts instead of a dedicated cost readout: ZeroRouter's bin lookup on mean success logit (ten quantile bins), and a ridge regression of log length on the success logits. Both come within a few points of the cost readout on LCB (ours saves 5.7% [−1.1, 13.4] and 3.2% [−2.8, 10.8]) and on Omni-MATH (5.8% [−2.8, 13.1] and −3.0% [−14.7, 3.9]), where output length tracks difficulty, and lose by 16.7% [10.8, 21.8] and 18.4% [11.0, 23.5] on MMLU-Pro. There, difficulty explains log length only in part, and even difficulty and subject together price worse than the prefill (Section 6). The prefill encodes length information beyond difficulty, and that information pays where difficulty and length come apart.

Representation, readout and prefill size. To compare representations at equal size, Table 9 reads cost from Qwen3-Embedding-4B (Zhang et al., 2025), a text-embedding model with the prefill's parameter count, with three readouts (our ridge recipe, a MixLLM-style ensemble and CARROT-style k-nearest neighbours) and our success predictions, and builds a fully text-based router that also predicts success from the embedding with our logistic recipe. On LCB, Omni-MATH, MMLU-Pro and AIME our readout saves 9.7–23.1% relative to every embedding cost readout and 6.8–18.5% relative to the text router, with intervals excluding zero (ahead in every training set against the cost readouts, and in 8–10 of 10 against the text router). On the other five pools it is ahead of every embedding cost readout on average and never significantly behind any arm (the text router is 4.7 points cheaper on CodeContests, within noise). The embedding's success predictions are weaker on every pool (AUC .73–.75 against .83 on LCB and Omni-MATH). We do not charge the embedding's own encoder pass, which would only favour our readouts. On PCA-reduced 4B prefill features the three readouts are within about 7 points of one another (manifest split), so the gain comes from the representation rather than the regressor. Table 16 varies the prefill model while keeping prompts, layers and both readouts fixed (ten training sets). Every Qwen3 size from 0.6B to 4B saves 17.0–20.6% on MMLU-Pro and 27.9–29.0% on Omni-MATH against median pricing with its own success readouts, and length R^2 rises with size (MMLU-Pro .33, .40, .44; Omni-MATH .61, .65, .66). The result is not specific to Qwen: prefills from three other families (Phi-4-mini (Microsoft, 2025), Granite-3.3-2B, SmolLM2-1.7B (Allal et al., 2025)) save 16.9–21.4% and 25.1–28.2%. None beats our prefill on average: ours saves 2.0–8.2 points relative to each of them, with pooled intervals excluding zero against Qwen3-0.6B, Granite and SmolLM2 on MMLU-Pro and including zero otherwise. Even the same-size Qwen3-4B hybrid release trails by 3.4 and 3.6 points on average (intervals including zero). Any small prefill works; size and release change the saving by a few points.

Is fine-tuning needed? A router could instead train a small model end to end on the length labels, as CARROT does with RoBERTa (Somerstep et al., 2025). We fully fine-tuned three such models on the same training problems and labels (DistilBERT 66M, jina-embeddings 137M and Qwen3-0.6B; learning rate and epoch chosen on calibration error) and used each as a cost estimator with our success predictions. On the

manifest split, each saves less than our readout on Omni-MATH and MMLU-Pro: ours saves 11.9–25.6% relative to each, with intervals excluding zero. LCB is the exception for the largest model: there ours saves 11.7–14.9% relative to the two smaller encoders, but averaged over ten training sets (each fine-tuned anew) only 6.5% [−1.2, 12.2] relative to the fine-tuned Qwen3-0.6B, ahead with an interval excluding zero in 7 of 10 training sets. Fine-tuning does not recover more than a linear readout of the frozen model: the same Qwen3-0.6B, frozen and read linearly, saves 31.9% and 22.0% relative to median pricing on Omni-MATH and MMLU-Pro (manifest split), more than its fine-tuned copy (20.0% and 19.3%), despite pairing with its own weaker success readouts (length R^2 .62 against .53 on Omni-MATH, .33 against .38 on MMLU-Pro). The length information is already present in a pretrained decoder's prefill, and a linear readout recovers it from activations the router already computes, without training a model per router.

Is the prefill router's success pipeline needed? The prefill router (Varshney et al., 2026) selects a layer, a pooling and a PCA dimension per target by cross-validation over the encoder's upper layers, concatenates the selected features and trains an ensemble of shared-trunk MLPs. We reimplemented this pipeline on the same encoder, splits and cost readout, and compared it with our per-route logistic readouts on fixed layers. On the manifest split the two are tied on all three test sets: ours saves −0.1% [−7.7, 4.0] on LCB, 1.1% [−1.0, 3.0] on MMLU-Pro and −2.2% [−5.7, 1.9] on Omni-MATH relative to it (success AUC .857 vs. .839, .715 vs. .714 and .829 vs. .842). On the two heterogeneous pools the comparison splits: ours saves 8.4% [−2.5, 18.4] relative to their pipeline on BIG-Bench Extra Hard, and their pipeline is ahead by 6.0% [0.4, 11.2] on SuperGPQA, where success is barely legible to either (AUC .58 and .57). Linear readouts on a fixed set of layers are enough on four of five sets, and they make the cost readout a drop-in addition: the same features feed both. They are also cheaper to fit: on 16 CPU threads, our success and cost readouts for five routes take about 3 minutes on LCB and MMLU-Pro (nine regularisation values per route, chosen on calibration), against about 7.4 minutes for the reimplemented pipeline (48 feature configurations × 5 folds per route, then ten networks); neither needs an encoder pass beyond the shared prefill, unlike the MixLLM-style and ZeroRouter estimators, which fit in seconds but read their own text encoder.

11 Deployment

Routing versus single models. Any point on the convex hull of the routes' cost–accuracy points can be reached without a router, by sending each query to a route drawn at random with fixed probabilities; this mix is the baseline a router has to beat (Hu et al., 2024). At matched accuracy our router spends 37.3% [29.5, 44.3] less than the mix on LCB, 39.2% [30.5, 45.8] on Omni-MATH and 22.9% [13.5, 28.8] on MMLU-Pro, and 14.9–46.6% less on AIME, SuperGPQA and BIG-Bench Extra Hard; it ties the mix on APPS, BigCodeBench and CodeContests. With median pricing, the same success readouts beat the mix by 10.5–29.1% on LCB, Omni-MATH, SuperGPQA and BIG-Bench Extra Hard, but not on MMLU-Pro (−0.7% [−5.7, 2.8]) or AIME (0.0%): there the cost estimate decides whether routing pays at all. The router does not beat the best single model at that model's accuracy. On every pool deepseek-v4-flash is the most accurate route or within 1.6 points of it, at 17–37% of the cost of gpt-oss-120b at high effort (Table 2). The router's most accurate policy, which sends each query to the route with the highest predicted success, ends 0.2–2.7 points below the most accurate route on seven pools (it ties on AIME and is 2.0 points above on BIG-Bench Extra Hard), and where it reaches deepseek-v4-flash's accuracy it spends as much as calling it alone (AIME, SuperGPQA, BIG-Bench Extra Hard) or more (the coding pools). Close to the top, routing buys little over calling deepseek-v4-flash alone: giving up one point of accuracy saves 5.1% on AIME and 9.4% on SuperGPQA (intervals excluding zero) and nothing significant elsewhere, while giving up five points saves 14.9–40.3% on the six pools other than APPS, BigCodeBench and CodeContests (LCB 32.6%, Omni-MATH 33.1%, MMLU-Pro 26.9%, BIG-Bench Extra Hard 40.3%), 1.5–1.8× what median pricing and 1.5–3.6× what a random mix of routes saves at the same accuracy. Below the top it is much cheaper than every other route at that route's accuracy: on the same six pools, 45–83% cheaper than gpt-oss-20b at medium effort and, where it reaches its accuracy, 50–92% cheaper than gpt-oss-120b at high effort. The shortfall at the top is not specific to deepseek-v4-flash. In the pool of Appendix I without it or gpt-oss-120b, the most accurate route, Nemotron-3-Super-120B, costs $2.1–7.5 per thousand calls and the router again ends 1.6–3.2 points below it; there our router saves 44.7% [36.6, 52.2] relative to the mix on LCB and 22.7% [1.9, 38.3] on Omni-MATH, against 21.4% and 10.9% with median pricing, and on MMLU-Pro neither beats

the mix (ours −7.6% [−39.3, 20.5], median pricing −39.9%). Under a hard per-query budget the router beats calling deepseek-v4-flash alone only at the tightest cap we tested (by 8.2 and 7.0 points of accuracy on LCB and MMLU-Pro); at looser caps it ties or trails it by up to 3 points, although knowing each problem's length would leave room (Appendix L).

Routing versus cascades. A cascade calls routes in sequence and stops when an answer is accepted (Chen et al., 2023; Aggarwal et al., 2023). To bound what any cascade could achieve on our pools, we give it a perfect judge: every ordered plan of the five routes is called cheapest first, each attempt is judged by its true outcome at no cost, and the cascade stops at the first correct answer. A learned judge can only do worse. Even against this bound, one-shot routing with our readouts (manifest split) spends 29.7% [19.6, 37.7] less at matched accuracy on LCB and 31.9% [26.4, 37.3] less on Omni-MATH, because a cascade pays for every failed attempt before the one it accepts. On MMLU-Pro the two tie (−0.8% [−6.1, 3.9]): the cheapest route solves 62% of questions at about 6% of the most expensive route's cost, so failed first attempts are cheap.

Deployable policies. Our main metric compares cost–accuracy frontiers on the test set; a deployed router must fix its trade-off V in advance. For each estimator and each of ten accuracy targets spanning the band every estimator reaches on the calibration split, we choose the cheapest V that reaches the target on calibration and apply it once to the test set. We compare its test cost with the median rule's deployed policies, interpolated at the accuracy it actually reaches (Table 17; ten training sets). Our deployed policies save 19.6–37.9% on LCB, Omni-MATH, MMLU-Pro, AIME, SuperGPQA and BIG-Bench Extra Hard; they cost 1.0–3.3% more than our own test frontier at the accuracy they reach and miss their target by 2.2–5.3 points. They lead the best deployed external estimator on LCB, Omni-MATH and MMLU-Pro with intervals excluding zero in every training set, on AIME in 9 and SuperGPQA in 7 of 10, and tie it on BIG-Bench Extra Hard; they lead pricing from success on MMLU-Pro and BIG-Bench Extra Hard in every training set and on SuperGPQA in 7. Deployed savings exceed the frontier savings mainly because median pricing deploys less well: its calibration-chosen policies fall 8.6–16.5% short of its own test frontier on MMLU-Pro, AIME, APPS, BigCodeBench, CodeContests and SuperGPQA, against at most 9.4% for ours. On APPS, BigCodeBench and CodeContests our lead is within noise.

A frontier reference. To place the pool against a frontier model, we called Claude Opus 5.5 (adaptive thinking at high effort, one sample per problem, same prompt and grader) on 100 random LiveCodeBench test problems. It solves 94.0% at $21.9 per thousand calls; it thinks on only 24% of these problems and writes 954 output tokens on average. On the same problems deepseek-v4-flash alone reaches 88.8% at $0.8 per thousand calls, 94% of the frontier model's accuracy at 4% of its cost, and gpt-oss-120b at high effort 89.3% at $3.4; our router's most accurate policy reaches 87.7% at $1.6 (mean over the ten training sets), no better than deepseek-v4-flash alone, as on the full test set. The gap to the frontier model is a property of the pool's strongest open route, not of routing; the router's saving lies below that route's accuracy.

The router's own cost. The features come from one forward pass of Qwen3-4B over the prompt: 127–1,216 tokens on average per pool, 19–28 ms median latency on one H100 at batch size one. A router that predicts success from the prefill already pays for this pass, so the cost readout adds nothing to it. Against no routing at all, the pass is not free when the routed models are as cheap as ours (well under a dollar per thousand queries). Self-hosted at an assumed $2 per H100-hour with unoptimised batched inference, it costs 2–10% of our deployed saving on Omni-MATH, AIME, MMLU-Pro and SuperGPQA and about 30% on BIG-Bench Extra Hard and APPS, whose prompts are long; at the input price of the smallest Qwen3 model listed on OpenRouter (14B, $0.12 per million tokens), it costs 6–51% of the saving, and more than the saving on APPS. Its share falls as the routed models get more expensive.

12 Discussion and Limitations

When does per-query cost pay? It pays when output length varies within a model more than prices differ between models: here lengths span 11–121× within a route (Fig. 1) while per-call costs span 17–30× across routes (Table 2), leaving 40–58% headroom over median pricing. In practice the saving comes from deciding, query by query, whether a strong route whose cost varies widely is worth calling instead of a cheap

one (Section 5). It is captured when an estimator can read that variation from the prompt; for reasoning routes it is mostly fixed by the problem (Section 6), so what limits capture is the estimator, most of all on route differences and on knowledge questions. Where length tracks difficulty (LCB, Omni-MATH), even difficulty-only pricing captures most of it; where it tracks the work a question requires (MMLU-Pro), only a dedicated readout does, and that is also where we capture least (40%), leaving the most room for better cost predictors.

Generalisation. In-domain and with full training data, a router could price from difficulty and lose little on most pools. The case for a dedicated cost readout is that it holds up where a deployed router actually operates: with few labels, with new models and on new benchmarks, where both difficulty-only pricing and text estimators lose most of their saving (Section 7). Transfer has limits: on Omni-MATH and BIG-Bench Extra Hard the transferred router saves little (4.0% and −3.4%), so a benchmark unlike its sources still needs some labels; on BIG-Bench Extra Hard, twenty labelled target problems added to the source data restore 24.1% (manifest split).

Limitations. Each route needs output-length labels, though a few dozen suffice for most pools. At the top of the frontier the router does not beat the most accurate single route, because its success predictions are imperfect (Section 11); its gains lie below that accuracy, and per-query budgets change this only at the tightest caps (Appendix L). Telling apart two strong routes of similar price is the open part of the problem: when we bill gpt-oss-120b at high effort at deepseek-v4-flash's token rates (keeping its own lengths and outcomes), perfect per-query cost knowledge would save 11–36% within that pair, but our readouts save at most 7.0% [2.4, 14.0] (Omni-MATH) relative to calling deepseek-v4-flash alone at its accuracy, and nothing significant on most pools. The saving also needs a cheap route that solves a good share of problems (Section 5). The main experiments use five routes from two model families; three further families were added on three sets (Section 7), and in the full pool the router needed none of them. Transfer is evaluated across benchmarks that share the routed models. Intervals are pointwise and assume independent problems; they include the choice of training problems only through ten training sets, and a few analyses that change the protocol use one training set. On Omni-MATH and MMLU-Pro the test set is one fixed sample of each benchmark with one draw per problem and route. Only deepseek-v4-flash is pinned to a provider; the gpt-oss routes are not, since their provider changes price but barely length. The prefill comparison covers four model families at up to 4B, and the text-embedding comparison one embedding model at the prefill's size. Billed rates were measured over one collection, and prices change within days. The ZeroRouter, MixLLM and CARROT baselines are reimplementations.

13 Conclusion

A frozen prefill that already predicts success also predicts cost. Per-query cost is worth predicting for reasoning models, whose lengths vary within a model by more than prices differ between models, and much less for non-reasoning ones. On held-out test problems at billed prices, prefill cost readouts reduce spending at matched accuracy and outperform every alternative estimator we tested; pricing from difficulty alone matches them only where output length tracks difficulty and only with full in-domain data. With few labels and for new models the dedicated readout keeps most of its saving, and on four of seven new benchmarks it does so without any target labels; in mixed traffic it adds a tenth of the cost beyond what knowing each query's benchmark and difficulty saves. Endpoints of one model share these readouts up to an offset, so the readouts can absorb provider drift from a few dozen calls, though pinning a provider is simpler.

Reproducibility

Every number in the paper is computed from stored per-call records (prompt, response, grade, token counts, billed cost and serving provider) and stored prefill activations, with the splits fixed by manifests; no analysis issues a new model call. Appendix D gives the evaluation protocol, Appendix C the reimplemented estimators, and Appendix A the pools, splits and billed rates. Collection used OpenRouter with deepseek-v4-flash

pinned to one provider; prices and provider behaviour change over time, so rerunning collection will not reproduce billed costs exactly, while every analysis reproduces from the released records.

TODO: Confirm what is released (code, per-call records, prefill features) and add the anonymised link.

References

Pranjal Aggarwal, Aman Madaan, Ankit Anand, Srividya Pranavi Potharaju, et al. AutoMix: Automatically Mixing Language Models, 2023. arXiv:2310.12963.

Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, et al. SmolLM2: When Smol Goes Big – Data-Centric Training of a Small Language Model, 2025. URL https://arxiv.org/abs/2502.02737. arXiv:2502.02737.

Ibragim Badertdinov, Alexander Golubev, Maksim Nekrashevich, et al. SWE-rebench: An automated pipeline for task collection and decontaminated evaluation of software engineering agents. *arXiv preprint arXiv:2505.20411*, 2025.

Dylan Bouchard. Is Escalation Worth It? On the Depth of LLM Cascades, 2026. URL https://arxiv.org/abs/2605.06350. arXiv:2605.06350.

Qi Cao, Shuhao Zhang, Ruizhe Zhou, et al. Models Under SCOPE: Scalable and Controllable Routing via Pre-hoc Reasoning, 2026. URL https://arxiv.org/abs/2601.22323. arXiv:2601.22323.

Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance, 2023. arXiv:2305.05176.

DeepSeek-AI. DeepSeek-V4-Flash, 2026. Model card, https://huggingface.co/deepseek-ai/DeepSeek-V4-Flash.

Jasper Dekoninck, Maximilian Baader, and Martin Vechev. A Unified Approach to Routing and Cascading for LLMs, 2024. arXiv:2410.10347.

di-zhang-fdu. AIME 1983–2024, 2024. Dataset, https://huggingface.co/datasets/di-zhang-fdu/AIME_1983_2024.

Tao Feng, Yanzhen Shen, and Jiaxuan You. GraphRouter: A Graph-based Router for LLM Selections, 2024. arXiv:2410.03834.

Bofei Gao, Feifan Song, Zhe Yang, et al. Omni-MATH: A Universal Olympiad Level Mathematic Benchmark For Large Language Models, 2024. URL https://arxiv.org/abs/2410.07985. arXiv:2410.07985.

Irena Gao, Percy Liang, and Carlos Guestrin. Model equality testing: Which model is this API serving? In *International Conference on Learning Representations*, 2025.

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, et al. The Llama 3 Herd of Models, 2024. URL https://arxiv.org/abs/2407.21783. arXiv:2407.21783.

Michael Günther, Jackmin Ong, Isabelle Mohr, et al. Jina Embeddings 2: 8192-Token General-Purpose Text Embeddings for Long Documents, 2023. URL https://arxiv.org/abs/2310.19923. arXiv:2310.19923.

Liang He, Jingbo Wen, Yixiong Chen, Yue Yang, Qizhen Lan, Kangning Cui, and Xilu Wang. You cannot pick a provider from the price list: Market-aware routing for open-weight LLM inference, 2026.

Dan Hendrycks, Steven Basart, Saurav Kadavath, et al. Measuring Coding Challenge Competence With APPS, 2021. URL https://arxiv.org/abs/2105.09938. arXiv:2105.09938.

Qitian Jason Hu, Jacob Bieker, Xiuyu Li, et al. RouterBench: A Benchmark for Multi-LLM Routing System, 2024. URL https://arxiv.org/abs/2403.12031. arXiv:2403.12031.

Naman Jain, King Han, Alex Gu, et al. LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code, 2024. URL https://arxiv.org/abs/2403.07974. arXiv:2403.07974.

Mehran Kazemi, Bahare Fatemi, Hritik Bansal, et al. BIG-Bench extra hard. *arXiv preprint arXiv:2502.19187*, 2025.

Janghoon Lee. Most of the LLM Routing Gap Is Task Type, 2026. arXiv:2608.23023.

Hao Li, Yiqun Zhang, Zhaoyan Guo, et al. LLMRouterBench: A Massive Benchmark and Unified Framework for LLM Routing, 2026a. arXiv:2601.07206.

Haorui Li et al. When is the same model not the same service? a measurement study of hosted open-weight LLM APIs, 2026b.

Yujia Li, David Choi, Junyoung Chung, et al. Competition-Level Code Generation with AlphaCode, 2022. URL https://arxiv.org/abs/2203.07814. arXiv:2203.07814.

Guanjie Lin, Yinxin Wan, Shichao Pei, Ting Xu, Kuai Xu, and Guoliang Xue. Behavioral consistency and transparency analysis on large language model API gateways, 2026.

William Lugoloobi, Thomas Foster, William Bankes, and Chris Russell. LLMs Encode Their Failures: Predicting Success from Pre-Generation Activations, 2026. arXiv:2602.09924.

M-A-P Team. SuperGPQA: Scaling LLM evaluation across 285 graduate disciplines. *arXiv preprint arXiv:2502.14739*, 2025.

Glen Messenger. ALPS: Activation-Based Length Prediction for Intelligent LLM Inference Scheduling, 2026. URL https://doi.org/10.5281/zenodo.19078431. Zenodo. doi:10.5281/zenodo.19078431.

Microsoft. Phi-4-Mini Technical Report: Compact yet Powerful Multimodal Language Models via Mixture-of-LoRAs, 2025. URL https://arxiv.org/abs/2503.01743. arXiv:2503.01743.

Nebius. SWE-agent trajectories, 2024. Dataset, https://huggingface.co/datasets/nebius/SWE-agent-trajectories.

Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, et al. RouteLLM: Learning to Route LLMs with Preference Data, 2024. arXiv:2406.18665.

OpenAI. gpt-oss-120b & gpt-oss-20b Model Card, 2025. URL https://arxiv.org/abs/2508.10925. arXiv:2508.10925.

Zhihong Pan, Kai Zhang, Yuze Zhao, and Yupeng Han. Route to Reason: Adaptive Routing for LLM and Reasoning Strategy Selection, 2025. arXiv:2505.19435.

Shivam Patel, Akaash R. Parthasarathy, Ankur Mallick, and Gauri Joshi. Beyond Accuracy and Cost: Latency-Aware LLM Query Routing for Dynamic Workloads, 2026. arXiv:2607.18253.

Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter, 2019. URL https://arxiv.org/abs/1910.01108. arXiv:1910.01108.

Seamus Somerstep, Felipe Maia Polo, Allysson Flavio Melo de Oliveira, et al. CARROT: A Cost Aware Rate Optimal Router, 2025. URL https://arxiv.org/abs/2502.03261. arXiv:2502.03261.

Seongho Son, Sangwoong Yoon, Jiahua Tang, Shuhan Wang, et al. SWE-Router: Routing in Multi-turn Agentic Software Engineering Tasks, 2026. arXiv:2607.00053.

Wei Song, Zhenya Huang, Cheng Cheng, Weibo Gao, et al. IRT-Router: Effective and Interpretable Multi-LLM Routing via Item Response Theory, 2025. arXiv:2506.01048.

Ihor Stepanov, Aleksandr Smechov, Mykhailo Shtopko, et al. SCX Router: Streaming Zero-Shot Model Selection with a Decoder-KV Classifier and a Real-World Task Ontology, 2026. URL https://arxiv.org/abs/2609.02292. arXiv:2609.02292.

Tanay Varshney, Annie Surla, Michelle Xu, et al. LLM Router: Rethinking Routing with Prefill Activations, 2026. URL https://arxiv.org/abs/2603.20895. arXiv:2603.20895.

Wenhui Wang, Furu Wei, Li Dong, et al. MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers, 2020. URL https://arxiv.org/abs/2002.10957. arXiv:2002.10957.

Xinyuan Wang, Yanchi Liu, Wei Cheng, et al. MixLLM: Dynamic Routing in Mixed Large Language Models, 2025. URL https://arxiv.org/abs/2502.18482. arXiv:2502.18482.

Yubo Wang, Xueguang Ma, Ge Zhang, et al. MMLU-Pro: A More Robust and Challenging Multi-Task Language Understanding Benchmark, 2024. URL https://arxiv.org/abs/2406.01574. arXiv:2406.01574.

Weihuang Wen, Yingying Liu, Yichuan Liu, et al. OUTLETS: Output-Length Prediction from Speculative Decoding Backbones, 2026. URL https://arxiv.org/abs/2609.01068. arXiv:2609.01068.

Huanyi Xie, Yubin Chen, Liangyu Wang, et al. Predicting LLM Output Length via Entropy-Guided Representations, 2026. URL https://arxiv.org/abs/2602.11812. arXiv:2602.11812.

Jiaqi Xue, Qian Lou, Jiarong Xing, et al. R2-Router: A New Paradigm for LLM Routing with Reasoning, 2026. URL https://arxiv.org/abs/2602.02823. arXiv:2602.02823.

Cheng Yan, Wuyang Zhang, Zhiyuan Ning, et al. Breaking Model Lock-in: Cost-Efficient Zero-Shot LLM Routing via a Universal Latent Space, 2026. URL https://arxiv.org/abs/2601.06220. arXiv:2601.06220.

An Yang, Anfeng Li, Baosong Yang, et al. Qwen3 Technical Report, 2025a. URL https://arxiv.org/abs/2505.09388. arXiv:2505.09388.

John Yang, Kilian Lieret, Carlos E. Jimenez, et al. SWE-smith: Scaling data for software engineering agents. *arXiv preprint arXiv:2504.21798*, 2025b.

Yanzhao Zhang, Mingxin Li, Dingkun Long, et al. Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models, 2025. URL https://arxiv.org/abs/2506.05176. arXiv:2506.05176.

Terry Yue Zhuo, Minh Chien Vu, Jenny Chim, et al. BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions, 2025. URL https://arxiv.org/abs/2406.15877. arXiv:2406.15877.

A Pools

B Every estimator on every pool

C Reimplemented estimators

Every estimator predicts each route's log mean output length from information available before generation, is fitted on the training problems only, and is post-processed like our readout (residual smearing, level matched to the route's training mean); configurations are chosen on the calibration split. *Prompt-feature GBM* follows latency-aware routing (Patel et al., 2026): histogram gradient boosting (300 iterations, learning rate 0.05, minimum leaf 10) on 44 prompt statistics (length, line count, count and magnitude of numbers, math delimiters, examples, constraint markers, code fences, inequality signs and 34 keyword counts), with scikit-learn's histogram boosting in place of LightGBM. *MixLLM-style* (Wang et al., 2025): a jina-embeddings-v2 137M embedding (Günther et al., 2023) of the prompt (the code variant on LiveCodeBench), read by the average of an MLP, a random forest and a k-nearest-neighbour regressor per route; MixLLM used a BERT-family encoder. *CARROT-style kNN* (Somerstep et al., 2025): the authors' local variant, MiniLM-L12 (Wang

Pool	Train / calib. / test problems	Draws per route (train)	Draws per route (test)	Test acc. (%)	Median output per route	90th/10th pct. ratio
LiveCodeBench (Jain et al., 2024)	441 / 110 / 341	16/12/8/6/3	16/12/8/6/3	52–90	0.7–7.0k	11–121
Omni-MATH (Gao et al., 2024)	275 / 75 / 1,000	4/3/3/2/2	1/1/1/1/1	42–71	1.4–10.5k	10–126
MMLU-Pro (Wang et al., 2024)	550 / 150 / 6,500	4/3/3/2/2	1/1/1/1/1	62–86	0.2–1.6k	7–29
AIME 1983–2024 (di-zhang-fdu, 2024)	513 / 139 / 281	4/3/3/2/2	4/3/3/2/2	57–98	1.8–7.7k	4–16
APPS (Hendrycks et al., 2021)	489 / 123 / 377	1/1/1/1/1	1/1/1/1/1	58–86	0.6–10.3k	10–91
BigCodeBench (Zhuo et al., 2025)	544 / 136 / 421	16/12/8/6/3	16/12/8/6/3	45–53	0.4–2.1k	2–4
CodeContests (Li et al., 2022)	346 / 86 / 266	4/3/3/2/2	4/3/3/2/2	49–86	0.8–13.1k	5–37
SuperGPQA (M-A-P Team, 2025)	550 / 150 / 300	4/3/3/2/2	4/3/3/2/2	31–68	0.2–3.3k	7–25
BIG-Bench Extra Hard (Kazemi et al., 2025)	550 / 150 / 300	4/3/3/2/2	4/3/3/2/2	23–65	0.4–16.4k	14–26

Table 10: **Pools.** Split sizes (problems with a valid answer from every route), median number of draws per route (gpt-oss-20b low, medium; deepseek-v4-flash; gpt-oss-120b medium, high), the range of test accuracy over routes, the range over routes of the median output length per problem (tokens), and the range over routes of the ratio between a route's 90th- and 10th-percentile problem lengths. Omni-MATH and MMLU-Pro are trained on the 500- and 1,000-problem pools (Omni-MATH-500 and a 1,000-question subject-stratified sample) and tested on 1,000 and 6,500 further problems; the other pools are tested on their own held-out split, drawn at random. Effective billed rates (USD per million input / output tokens): gpt-oss-20b 0.029 / 0.134, deepseek-v4-flash (StreamLake) 0.042 / 0.084, gpt-oss-120b 0.020 / 0.257.

	External estimators					Our ablations	
Pool	Median	Mean	Prompt GBM	MixLLM-style	ZeroRouter	Difficulty bins	From success
LiveCodeBench	**27.8**	**27.8**	**20.7**	**16.2**	**19.3**	5.7	3.2
Omni-MATH	**31.5**	**34.3**	**25.8**	**24.7**	14.2	5.8	−3.0
MMLU-Pro	**23.4**	**25.8**	**20.0**	**15.5**	**18.4**	**16.7**	**18.4**
AIME	**14.9**	**15.0**	**14.2**	**13.5**	**8.3**	**7.9**	4.3
APPS	**16.8**	**13.3**	9.7	**16.4**	2.3	2.2	−2.2
BigCodeBench	0.4	3.8	1.4	1.7	−10.7	2.4	2.4
CodeContests	**9.4**	**9.4**	8.0	4.3	−1.0	1.7	−0.8
SuperGPQA	**33.0**	**35.8**	**19.1**	**15.7**	**35.2**	**25.1**	15.3
BIG-Bench Extra Hard	**23.5**	**22.7**	2.1	−1.3	**24.5**	**14.2**	**22.6**

Table 11: **Cost saved (%) by our readouts relative to each estimator** at matched accuracy on each pool's test set, mean over ten training sets; billed costs, deepseek-v4-flash pinned, same success predictions for every arm except ZeroRouter (its own). Bold: 95% paired bootstrap interval, pooled over training sets, excludes zero. No external estimator is significantly cheaper than ours on any pool. Difficulty bins: ZeroRouter's ten-bin lookup on our mean success logit; from success: ridge of log length on our success logits and their squares.

et al., 2020) sentence embeddings with uniform cosine kNN, k chosen by five-fold cross-validation on training problems. *ZeroRouter* (Yan et al., 2026): stage 1 fits a D-dimensional two-parameter item-response model to the five routes' training outcomes (MAP with Gaussian priors; the paper uses stochastic variational inference over about 200 leaderboard models); stage 2 regresses each problem's item parameters $(\log \alpha, b)$ on our 4B prefill features (PCA to 256 dimensions, ridge), a stronger reader on our pools than its fine-tuned DistilBERT (Sanh et al., 2019), which we also ran; cost is the route's mean training length within K quantile bins of $s = \alpha^\top b$. $D \in \{1, 5\}$ and $K \in \{5, 10, 20\}$ are chosen on calibration. With five routes a higher-dimensional latent does not improve its success predictions, and on MMLU-Pro adding up to 53 Open LLM Leaderboard models to stage 1 did not either. In a component swap during development, ZeroRouter's success model paired with our cost readout came close to our router, while our success readouts paired with its bin pricing did not: its shortfall is in the pricing. The fine-tuned encoders of Section 10 are mean-pooled with a linear head on per-route standardised log length, trained with masked squared error (AdamW, 6% warm-up then linear decay, maximum 512 tokens for DistilBERT and 2,048 for the others); learning rate (two values per encoder) and epoch (up to 10, or 20 when the best epoch was the last) are chosen on calibration error, and smearing and level match use the calibration problems.

D Protocol details

Labels and costs. Every draw is scored by the benchmark's own grader (code execution against the tests for the coding pools; answer matching for the others). A problem's realized cost on a route is the mean billed cost of its valid draws on the Omni-MATH and MMLU-Pro test sets, and the mean of its valid draws' input and output tokens at the effective billed rates elsewhere; evaluated problems need a valid answer from every route. Effective rates are least-squares fits of each call's billed cost on its input and output tokens, per model; Section 3 reports the check with rates fitted on training calls only. deepseek-v4-flash is pinned by restricting OpenRouter's provider routing to one provider with fallbacks disabled; the gpt-oss routes are served by whichever provider OpenRouter selects.

Readouts. Success readouts are per-route L2 logistic regressions on the standardised prefill features, with the regularisation chosen on calibration AUC from nine values and Platt scaling fitted on calibration. Cost readouts are per-route ridge regressions of log mean output length, with the regularisation chosen by cross-validation within the training problems.

Frontiers and matched accuracy. Each value V on a geometric grid of 300 values gives a deterministic policy (Eq. (1)) and a test point (mean realized cost, mean accuracy). An arm's frontier is the upper concave hull of its points, and its cost at a target accuracy is linearly interpolated on the hull. Two arms are compared at 12 accuracies evenly spaced over the interior 90% of the accuracy range both frontiers reach; the reported saving is one minus the geometric mean of their cost ratios. A training set reshuffles the training and calibration problems of the manifest (with a fixed seed per training set) into splits of the original sizes and keeps the test problems; the success readouts and every estimator are refitted and every configuration is re-chosen on its calibration problems. Within a training set, intervals resample test problems with replacement (300 paired resamples, the same for both arms and for every training set, all routes of a problem kept together) with that set's readouts fixed; reported intervals are percentiles of the 3,000 resamples pooled over the ten training sets, and reported values are means of the ten point estimates.

E Provider endpoints

Endpoints. The same model served by different providers behaves like the model plus an offset: pinning deepseek-v4-flash to each of three providers, they agree on correctness for 90–96% of problems and their output lengths correlate at .83–.95, but one writes about half as many tokens as another. Reusing the readouts with one multiplicative length offset per endpoint, estimated from 50 calls, removes the resulting cost bias on average, though a single estimate is noisy. A few dozen calls per provider also identify the cheapest one, and pinning it beats routing over providers; we therefore pin deepseek-v4-flash throughout, which also makes its length more predictable and raises our saving on every large set.

F Non-reasoning routes

Non-reasoning lengths do vary (s.d. of log length .53–1.73 per route), but the variation has two sources that the prompt predicts poorly. Llama-3.1-8B, at its default sampling settings, falls into repetition until the 16k cap on 9–12% of math problems (four in five capped answers loop), so its mean output is 4–6× its median and its log-length R^2 is below zero on every set. This is the model rather than the provider: redrawn on 300 such problems, it loops again on 29% with the same provider and 19% with another, against about 5% on control problems for both; a repetition penalty of 1.1 removes most loops (6%), apparently at some cost in accuracy. Qwen3-30B-A3B-Instruct, asked for code only on LCB, reasons inside code comments on hard problems until the cap (9% of problems), so its lengths span 142× between its 10th- and 90th-percentile problems (R^2 .27): the largest non-reasoning headroom comes from a non-reasoning model that reasons anyway.

G Further mechanism results

All results in this appendix are on the manifest split.

Success and cost are read from the same layers. Refitting both readouts on each stored layer separately, success AUC and length R^2 peak in the same late layers at the last token (layers 22–36 of 36), and one such layer matches the full concatenation (LCB AUC .86–.87, R^2 .73–.75, saving 33–34%; Omni-MATH .83–.84, .71–.72, 35–37%; MMLU-Pro .72, .48–.50, 27–29%). The two readouts are close to one signal where length tracks difficulty: the mean success logit correlates with predicted log length at $-.90$ to $-.97$ per layer on LCB and $-.71$ to $-.84$ on Omni-MATH, but only $-.36$ to $-.58$ on MMLU-Pro.

What changes in the routing. At matched accuracy in the middle of the band, 31–66% of problems change route relative to median pricing. The dominant changes are between gpt-oss-20b-low and deepseek-v4-flash: problems predicted to be short move to the more accurate deepseek-v4-flash (19–38% of problems), and problems predicted to be long move to the cheap route (9–18%).

The part of predicted length that difficulty does not explain. The part of the predicted length that the true solve rates do not explain is real. It correlates with the same part of the realized length at .56–.92 across six pools (MMLU-Pro .81: engineering questions are predicted 0.95 log units longer than their difficulty implies, and are 0.95 longer). It also predicts the length of five held-out non-reasoning models on MMLU-Pro (partial correlation .40–.71) while saying almost nothing about whether the stronger of them succeed ($|r| \leq .05$ for deepseek-v4-flash without thinking and Qwen3-235B-Instruct).

H What the data fix, and what our readout reads

Table 12 gives, per route, three test R^2 values of log mean output length: our readout's, which is the highest of every pre-generation estimator we evaluated (pricing from success, ZeroRouter's latent, prompt features, MixLLM-style and nearest-neighbour readouts of the text encoder of Appendix C, ridge, MixLLM-style and nearest-neighbour readouts of Qwen3-Embedding-4B, and on LCB the fine-tuned Qwen3-0.6B) on every route of every pool; that of a linear map from the mean realized log length of the other four routes on the same problem, which is not a deployable estimator but shows how much of a route's length is shared with other models; and the noise ceiling, the share of the variance of a route's per-problem log length over test problems that is between problems rather than between draws (from the routes' repeated draws; APPS has one draw per problem). Means over ten training sets.

Table 13 decomposes log length with no predictor. Over problems, routes and draws (problems with a valid draw on every route), after removing route means, a two-way random-effects decomposition (method of moments, harmonic-mean draw count) splits the variance into a shared problem level, a problem-by-route interaction and within-cell noise. Headroom by source compares oracles that know, for each test problem, its realized shared level only (with each route's training-mean offset), its route-specific part only (with the training-mean level), or both (the headroom of the main text); each is level-matched to the route's training mean like the readouts.

We fixed four models of headroom in advance, fitted by least squares over the 138 route subsets of Appendix J (full pools, leave-one-out subsets and pairs), using only predictor-free statistics: (A) the price spread (s.d. over routes of log mean cost per call), (B) the problem and interaction variances, (C) both, and (D) the log ratio of query-driven variance to squared price spread. In-sample R^2 is .32, .18, .37 and .11; leave-one-pool-out R^2 is .25, $-.23$, .01 and $-.01$. Headroom depends on which routes solve which problems, not only on how lengths and prices vary.

I Three further model families

Three further model families. We added Qwen3-32B, GLM-4.7-Flash and Nemotron-3-Super-120B, all reasoning, on LCB, Omni-MATH and a 2,700-problem subset of MMLU-Pro (billed \$0.40–0.57 per million

Pool	20b low	20b med.	deepseek	120b med.	120b high
LiveCodeBench	.74 / .85 / .99	.71 / .96 / .99	.72 / .92 / .99	.73 / .95 / .99	.75 / .94 / .98
Omni-MATH	.68 / .60 / .84	.66 / .86 / .89	.71 / .79 / .95	.72 / .87 / .94	.73 / .88 / .95
MMLU-Pro	.61 / .42 / .71	.52 / .68 / .79	.26 / .62 / .92	.62 / .76 / .88	.45 / .67 / .89
AIME	.41 / .52 / .89	.44 / .85 / .93	.50 / .79 / .97	.49 / .86 / .92	.51 / .87 / .94
APPS	.56 / .69 / –	.54 / .80 / –	.63 / .86 / –	.63 / .88 / –	.68 / .88 / –
BigCodeBench	.69 / .66 / .97	.61 / .86 / .96	.47 / .71 / .95	.71 / .85 / .97	.45 / .67 / .88
CodeContests	.46 / .67 / .92	.35 / .83 / .93	.52 / .80 / .96	.42 / .88 / .94	.43 / .86 / .95
SuperGPQA	.75 / .50 / .94	.47 / .64 / .88	.22 / .46 / .96	.63 / .77 / .92	.34 / .54 / .90
BIG-Bench Extra Hard	.56 / −.01 / .81	.81 / .82 / .89	.83 / .72 / .96	.90 / .75 / .92	.84 / .69 / .94

Table 12: **How much of each route's length we read, and how much the data fix.** Test R^2 of log mean output length: our readout / a linear map from the other routes' realized lengths on the same problem / noise ceiling (upper bound for any predictor of the problem). Mean over ten training sets; –: one draw per problem.

	Share of log-length variance (%)				Headroom by source (%)		
Pool	Problem	Interaction	Noise	Corr.	Level only	Route-specific only	Both
LiveCodeBench	84	8	8	.94	25.5	24.8	40.1
Omni-MATH	80	12	8	.85	23.3	41.4	51.9
MMLU-Pro	66	17	17	.70	25.4	43.7	58.0
AIME	70	15	16	.79	15.6	23.8	29.2
APPS	81	19	–	.86	18.3	24.6	34.4
BigCodeBench	57	15	28	.80	−0.4	8.7	10.0
CodeContests	73	13	14	.83	18.8	28.9	35.8
SuperGPQA	55	27	18	.67	43.3	54.1	60.6
BIG-Bench Extra Hard	42	29	29	.49	20.9	31.5	38.1

Table 13: **Where the routing value lies, without a predictor.** Left: variance of log output length after removing route means, split into the shared problem level, the problem-by-route interaction and run-to-run noise (APPS: one draw per problem, so the interaction includes noise); Corr.: mean correlation between two routes' mean log lengths over problems. Right: cost saved relative to median pricing by oracles that know each test problem's shared level only, its route-specific part only, or both (the headroom); mean over ten training sets.

output tokens; Qwen3-32B's provider stops at 24,575 output tokens, and Nemotron has no valid answer for 27% of Omni-MATH problems after retries), and fitted the same readouts for them (test success AUC .72–.87, log-length R^2 .23–.81). Averaged over ten training sets, with all eight routes our readouts save 28.8%, 25.6% and 23.0% relative to median pricing, beat difficulty bins (15.5 [6.8, 23.2]) and pricing from success (6.7 [0.3, 12.2]) on MMLU-Pro, and tie both on LCB and Omni-MATH, as with five routes. In the full pool the new models are never needed (removing any one changes the saving by under 0.2 points on the manifest split): deepseek-v4-flash is cheaper than each of them at a higher accuracy. We therefore also route in a pool of both gpt-oss-20b efforts and the three new models, without deepseek-v4-flash or gpt-oss-120b. There our readouts save 29.5% [17.3, 37.2], 12.4% [−0.2, 21.7] and 22.7% [9.9, 33.6] relative to median pricing on LCB, Omni-MATH and MMLU-Pro, beat difficulty bins on LCB (7.9 [0.3, 14.3]) and MMLU-Pro (12.5 [0.9, 23.6]) and tie pricing from success; with all routes except deepseek-v4-flash they save 25.3%, 11.4% and 22.4%. Without deepseek-v4-flash no single route is cheap and accurate: the most accurate one, Nemotron-3-Super-120B (85.2%, 75.2% and 82.4%), costs \$6.6, \$7.5 and \$2.1 per thousand calls against \$0.6, \$0.9 and \$0.2 for gpt-oss-20b at medium effort; our router's most accurate policy ends 1.6–3.2 points below it. With gpt-oss-120b in the pool, the router reaches Nemotron's accuracy on LCB at 64.8% [42.3, 83.8] lower cost. Onboarding is tested in the pool of cheap routes (ten training sets, 20 random choices of the k anchors in each). There only Nemotron matters (without it the saving falls from 29.5% to 21.1% on LCB, 12.4% to 10.5% on Omni-MATH and 22.7% to 8.1% on MMLU-Pro). Onboarded from 10 problems, our router saves 15.4%, 11.0% and 15.5%, while ZeroRouter's onboarding saves −35.1%, −30.2% and −6.6% and onboarding

at median length and base rate −77.3%, −49.0% and −26.7%. Ours leads ZeroRouter's onboarding by 50.4, 41.2 and 22.2 points on average, with intervals excluding zero in 6, 2 and 1 of the ten training sets; at k=50 by 32.4, 85.3 and 16.1 points, in 10, 8 and 6. A badly onboarded model can cost more than it adds: from 10 examples, onboarding at median length and base rate leaves the router spending more than median pricing for seven of the nine model–pool pairs, ZeroRouter's onboarding for six, ours for none. Finally (manifest split), the new models' readouts trained on Omni-MATH and used on MMLU-Pro keep the full saving (28.5% against 28.6% in-domain).

J Route subsets and length spread

		Without					Pair: gpt-oss-20b low and	
Pool	All five	20b low	20b med.	deepseek	120b med.	120b high	deepseek	120b high
LiveCodeBench	**27.8**	6.5	**28.4**	**22.4**	**27.8**	**28.3**	**29.0**	**32.1**
Omni-MATH	**31.5**	**16.2**	**32.6**	**15.2**	**31.8**	**32.0**	**33.8**	**22.4**
MMLU-Pro	**23.4**	–	**25.5**	**17.0**	**24.0**	**23.2**	**27.0**	**25.9**
AIME	**14.9**	−0.2	**14.9**	**7.0**	**15.0**	**14.9**	**15.0**	**10.7**
APPS	**16.8**	1.3	**15.0**	**11.5**	11.6	**19.3**	16.3	9.1
BigCodeBench	0.4	−45.0	1.8	3.8	0.8	1.8	2.9	6.2
CodeContests	**9.4**	4.3	**10.1**	**7.1**	9.0	**10.2**	**10.7**	11.2
SuperGPQA	**33.0**	0.7	**33.4**	15.4	**34.0**	**32.8**	**34.6**	24.1
BIG-Bench Extra Hard	**23.5**	3.8	**21.9**	**26.2**	**22.9**	**24.3**	**20.9**	**33.5**

Table 14: **Which routes the saving needs.** Cost saved (%) by our readouts relative to median pricing when the router is refitted on a subset of the five routes; mean over ten training sets; bold: pooled 95% paired bootstrap interval excludes zero. –: the two frontiers do not overlap (MMLU-Pro without gpt-oss-20b at low effort, where deepseek-v4-flash has the lowest median cost per call).

	s.d. of log output length					Price spread	R^2 beyond difficulty	
Pool	20b low	20b med.	deepseek	120b med.	120b high	between routes	deepseek	other routes
LiveCodeBench	0.90	1.34	**1.72**	1.18	1.40	1.07	0.10	0.09
Omni-MATH	0.88	1.34	**1.72**	1.14	1.47	0.99	0.07	0.14
MMLU-Pro	0.81	0.90	**1.29**	0.75	1.05	0.92	0.44	0.37
AIME	0.51	0.92	**1.06**	0.78	0.88	0.79	0.03	0.07
APPS	0.87	1.31	**1.71**	1.13	1.41	1.14	0.07	0.04
BigCodeBench	0.33	0.37	**0.60**	0.34	0.52	0.80	0.33	0.34
CodeContests	0.65	1.06	**1.29**	0.92	1.04	1.11	0.09	0.04
SuperGPQA	0.73	0.78	**1.23**	0.72	0.96	1.10	0.17	0.16
BIG-Bench Extra Hard	1.15	1.23	**1.26**	1.15	1.02	1.25	0.45	0.48

Table 15: **Length spread per route** (manifest split). s.d. of log per-problem output length over all problems (bold: widest on the pool); price spread: s.d. over routes of log mean cost per call; R^2 beyond difficulty: test log-length R^2 of our readout minus that of pricing from the success readouts (mean over the other four routes in the last column).

K Representation and prefill size

L Deployment

Per-query budgets. A per-query budget is the one setting where knowing when the strongest route will run long could beat calling it alone at any accuracy. We cap the cost of every call at X (a call that would exceed it is cut off and fails), with X at the 50th, 70th, 80th, 90th and 95th percentile of deepseek-v4-flash's cost per call on the training and calibration problems, and send each query to the route with the highest predicted probability of succeeding within the cap (ten training sets, full test sets). With our readout, that

	Omni-MATH			MMLU-Pro		
Prefill	R^2	vs. median	Ours vs. it	R^2	vs. median	Ours vs. it
Qwen3-0.6B	.61	27.9	7.3 [−0.6, 15.7] (5)	.33	17.0	**8.2 [0.5, 14.0]** (8)
Qwen3-1.7B	.65	29.0	4.1 [−5.5, 13.4] (4)	.40	19.4	5.2 [−1.5, 12.1] (7)
Qwen3-4B (hybrid)	.66	28.1	3.6 [−6.5, 12.3] (4)	.44	20.6	3.4 [−2.5, 7.4] (8)
Phi-4-mini (3.8B)	.64	28.2	7.6 [−2.4, 16.5] (7)	.43	21.4	2.0 [−6.2, 6.8] (5)
Granite-3.3 (2.5B)	.62	25.1	5.6 [−3.3, 14.6] (3)	.38	18.2	**7.1 [2.1, 12.8]** (10)
SmolLM2 (1.7B)	.58	25.2	5.9 [−3.7, 14.0] (6)	.33	16.9	**7.5 [2.1, 11.5]** (9)
Qwen3-4B-2507 (ours)	.70	31.5	–	.49	23.4	–

Table 16: **Other prefills.** Each prefill feeds both readouts, refitted with the paper's recipes on each of ten training sets; Omni-MATH and MMLU-Pro test sets, billed costs; means over training sets. R^2: test log-length R^2 averaged over routes. vs. median: cost saved (%) relative to median pricing with the same row's success predictions. Ours vs. it: cost saved by our router relative to the row's router, with the 95% paired bootstrap interval pooled over training sets (bold: excludes zero) and the number of training sets in which ours is ahead with an interval excluding zero; ours is significantly behind in no training set except one against Phi-4-mini on MMLU-Pro.

probability is the success readout times the chance that the route's length fits, from the predicted length and the training residuals. At the tightest cap the router beats calling deepseek-v4-flash alone on LCB (8.2 points of accuracy [5.8, 10.8]) and MMLU-Pro (7.0 [5.7, 8.3]), ahead in every training set, and the best single route by 5.3 and 0.9 points; on Omni-MATH it is 1.8 points ahead [−0.6, 4.3]. At looser caps it ties deepseek-v4-flash alone or falls 1–3 points behind (intervals excluding zero on MMLU-Pro and Omni-MATH at the 90th and 95th percentiles), at a spend between 19% below and 11% above deepseek-v4-flash's. Knowing each problem's realized length would beat deepseek-v4-flash alone by 9.3–15.5 points at the tightest cap and by 3.9–9.7 at the 70th percentile, so there is room under budgets; a readout trained directly on whether a call fits within the cap (a logistic readout per route and cap on the same prefill) reads it no better (within 2 points of ours everywhere, with the same AUC for whether deepseek-v4-flash fits, .81–.93). At the 95th percentile even known lengths do not beat deepseek-v4-flash alone, so there, too, the limit is the success readouts. Against median pricing under the same caps our readout is ahead by up to 12.6 points (LCB at the 90th percentile, where the median rule sends queries to routes whose median length fits the cap but whose long outputs do not), and behind significantly at no cap.

M Software-engineering agents

Agents: cost is real, varies by task and is stable. Public SWE-rebench trajectories (Badertdinov et al., 2025) give 111 tasks run five times by each of 13 models in one scaffold. Across models, agentic cost is dominated by the model: of the variance of log cost, 74% is between models, 15% between tasks, 6% their interaction and 4% run to run. Within a model, however, per-task cost spans 2.7–9.0× between the 10th and 90th percentile, and it is stable across runs (intraclass correlation .69–.92). Whether knowing it pays depends on the price gaps between the routed models, as in Section 8: among all 13 models, whose prices per run span about 190×, per-task cost knowledge is worth 8.5% [−5.0, 17.8] (one-shot routing, success and cost from three runs, scored on the other two); among the seven open-weight models, whose prices span about 10×, it is worth 25.7% [6.3, 40.6], as much as on the reasoning pools.

Agents: cost is only partly legible before the run. From the issue text alone, the prefill's cost readout reaches out-of-fold R^2 of about zero on the 111 SWE-rebench tasks, too few to fit it. On 3,387 tasks with about 20 SWE-agent runs each (one model, Llama-3.1-70B (Nebius, 2024); cost proxied by cumulative context and output length), the per-task mean cost is reliable (split-half reliability .91) and the issue text predicts it at R^2 .29, still rising with data (.18 with 100 tasks, .32 with 2,710). This is lower than what the prefill reaches on most one-shot pools (Table 12). Reading the first ten steps of a cheap model's trajectory

Pool	Ours vs median, deployed	Best external (deployed)	Ours − best external	Ours − cost from success	Ours: frontier loss / target miss
LiveCodeBench	31.3 (7.0)	MixLLM 14.9	16.3 (10)	2.7 (2)	2.4 / 2.6
Omni-MATH	35.0 (2.2)	ZeroRouter 13.9	21.5 (10)	−1.6 (0)	2.0 / 5.3
MMLU-Pro	31.1 (4.8)	kNN 18.1	12.7 (10)	17.3 (10)	1.0 / 2.3
AIME	19.6 (2.1)	ZeroRouter 10.3	9.4 (9)	2.2 (2)	1.3 / 4.6
APPS	23.3 (5.1)	ZeroRouter 15.8	5.2 (1)	−2.4 (0)	9.4 / 3.6
BigCodeBench	7.0 (5.2)	kNN 5.7	2.3 (0)	3.8 (1)	9.2 / 2.2
CodeContests	13.5 (4.4)	kNN 8.4	5.7 (2)	−0.3 (0)	6.3 / 5.6
SuperGPQA	37.9 (7.1)	MixLLM 24.6	14.1 (7)	15.4 (7)	3.3 / 2.5
BIG-Bench Extra Hard	24.0 (4.5)	MixLLM 24.5	−1.0 (0)	28.7 (10)	2.2 / 2.2

Table 17: **Deployable policies.** Each estimator picks, on the calibration split, the cheapest V reaching each of ten accuracy targets, applied once to the test set (same success readouts for all). Cost saved (%) relative to the median rule's deployed policies at the accuracy reached, mean over targets, then over ten training sets (in parentheses: s.d. over training sets for our saving; for the differences, the number of training sets in which ours is ahead with a 95% paired bootstrap interval over test problems excluding zero; ours is significantly behind in no training set on any pool except one APPS and one Omni-MATH set against pricing from success). Last column: extra cost over our own test frontier at the accuracy reached (%) / mean |test accuracy − target| (points). Billed prices.

predicts other models' per-task cost at R^2 .16 and the same run's final cost at .28, which suggests exploring cheaply before pricing the expensive candidates.

N Pre-registered predictions

Before collecting two of the pools, we recorded a forecast of whether the cost readout would pay there, from a cheap screen. The rule for Omni-MATH-500 was fixed before its screen: from one gpt-oss-20b-low draw per problem, if the prefill's cross-validated log-length R^2 was at least .50, predict a saving of at least 10% over median pricing with an interval excluding zero; if at most .35, predict none; in between, make no call. The screen gave .67, and the forecast (a gain, and headroom of at least 15%) was confirmed on the full pool at the time, on the manifest split: 21.9% [8.9, 33.5] at list prices with the pre-registered Thinking prefill, headroom 44.2%. The same rule run as a screen elsewhere predicted a gain on APPS, which was not significant before deepseek-v4-flash was pinned and is at pinned billed prices (16.8% [4.2, 30.4]) and wrong on AIME, where it predicted no gain and our readout saves 14.9% [10.8, 20.1]: a single draw of the cheapest route understates how predictable lengths are. A second screen, the gap between length read directly from the prefill and length read from a success readout, chose SuperGPQA and BIG-Bench Extra Hard as pools where the dedicated readout should beat pricing from success; it did on BIG-Bench Extra Hard (22.6 [12.3, 31.9]) and overstated the gap on SuperGPQA (15.3 [−0.6, 32.3]).

O Random re-splits and price gaps

Pool	Headroom	Ours vs median	Ours − best external	Splits ahead of all external	Ours − cost from success	Splits ahead / behind
LiveCodeBench	45.9 ± 3.0	34.1 ± 3.5	12.9 ± 2.9	10	2.2 ± 2.6	4 / 0
Omni-MATH	41.7 ± 2.5	19.0 ± 3.3	12.0 ± 3.8	9	−1.4 ± 2.6	1 / 2
MMLU-Pro	58.6 ± 1.0	25.8 ± 2.3	16.3 ± 2.6	10	12.8 ± 6.1	10 / 0
AIME	28.5 ± 2.7	15.3 ± 2.6	9.5 ± 2.6	10	2.5 ± 2.4	3 / 0
APPS	39.5 ± 6.0	15.6 ± 4.9	3.9 ± 2.8	0	−1.5 ± 2.4	0 / 0
BigCodeBench	11.0 ± 6.2	3.6 ± 2.3	−1.8 ± 4.7	0	4.2 ± 2.7	1 / 0
CodeContests	34.8 ± 2.0	8.7 ± 4.5	2.1 ± 4.0	0	−0.1 ± 3.3	0 / 0
SuperGPQA	59.0 ± 2.8	32.1 ± 5.8	5.8 ± 5.0	3	10.4 ± 7.6	5 / 0
BIG-Bench Extra Hard	37.6 ± 3.5	24.8 ± 5.7	1.0 ± 1.1	1	20.6 ± 5.3	10 / 0

Table 18: **Ten random re-splits per pool** (original train and calibration sizes; success readouts refitted with the paper recipe, every cost estimator refitted, configurations chosen on calibration). Mean ± s.d. over splits of cost saved (%) relative to median pricing, and of our saving relative to the best external estimator in that split (mean length, prompt GBM, MixLLM-style, CARROT-style kNN, ZeroRouter) and to pricing from success. "Splits ahead": splits where our 95% paired interval excludes zero against every external estimator, and (last column) ahead of / behind pricing from success. No external estimator is significantly cheaper than ours in any split. Unlike the training sets of the main results, these re-splits also reshuffle the test problems; on Omni-MATH and MMLU-Pro single-draw test problems move into training and multi-draw training problems into test. Values therefore differ from Table 3; the conclusions do not.

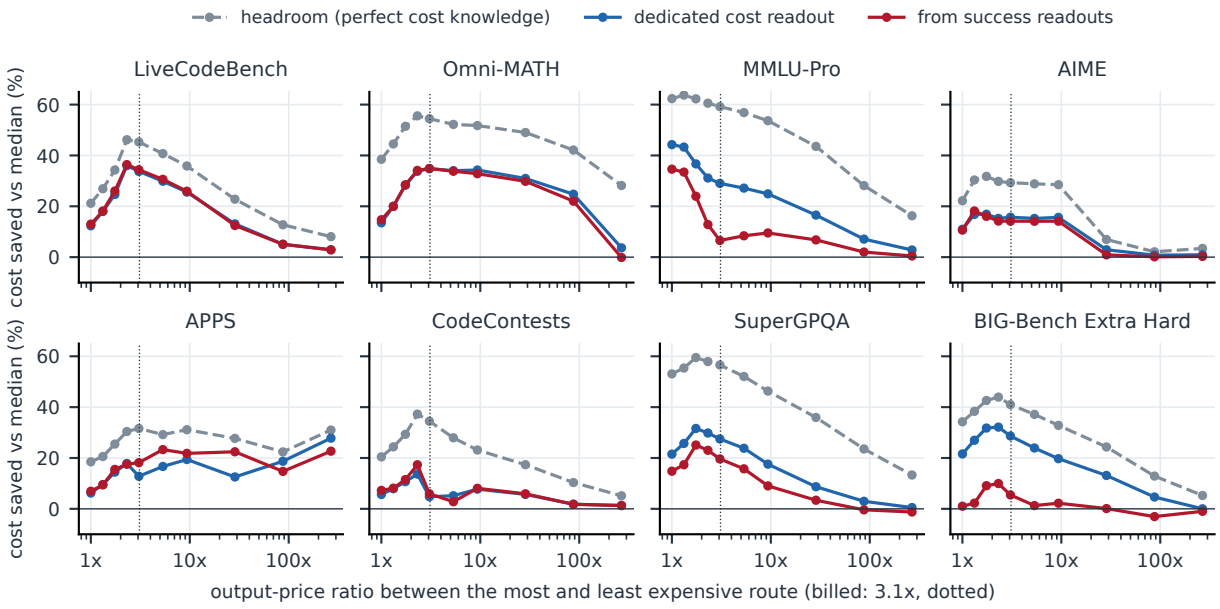


Figure 5: **Price gaps.** Cost saved relative to median pricing as every route's per-token price is moved toward or away from the geometric mean of the routes' prices (r_m^α, rescaled; α from 0, flat, to 5), with realized lengths fixed and costs recomputed from token counts (so values differ slightly from the billed-cost figures in the text). Dashed: headroom (perfect per-query cost knowledge); blue: our cost readout; red: pricing from the success readouts. Dotted: the billed spread (3.1×). BigCodeBench is omitted (its matched-accuracy band degenerates at some spreads).